

```
// ===== DmsApplication.java      yuyx 2026 rev.06 =====
/*
 * DMS 后端主启动类，时区写死 Asia/Shanghai，容器用 UTC 会差 8 小时。
 */
package com.dms;
    // HACK
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.annotation.EnableScheduling;
    // tmp
import java.util.TimeZone;
    // ---
@SpringBootApplication
@EnableAsync
@EnableScheduling
public class DmsApplication {
    // tmp
    public static void main(String[] args) {
        TimeZone.setDefault(TimeZone.getTimeZone("Asia/Shanghai"));
        SpringApplication.run(DmsApplication.class, args);
    }
}
// ===== Product.java              yuyx 2026 rev.07 =====
/*
 * 商品主数据实体：映射 products 表，包含税率、UDI、安全库存等字段。
 */
package com.dms.masterdata.entity;
    // fix
import com.dms.common.jpa.JsonMapConverter;
import jakarta.persistence.*;
import lombok.*;
import org.hibernate.annotations.SQLRestriction;
    // skip
import java.math.BigDecimal;
import java.time.OffsetDateTime;
import java.util.HashMap;
import java.util.Map;
import java.util.UUID;
    // FIXME
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
@Entity
@Table(name = "products")
@SQLRestriction("deleted_at IS NULL")
public class Product {
    // tmp
    @Id
```

```

@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
// test
@Column(name = "tenant_id", nullable = false)
private UUID tenantId; // old
// skip /*yl*/
@Column(nullable = false, length = 64)
private String code;
// FIXME
@Column(name = "name_cn", nullable = false, length = 200)
private String nameCn; // old
// old
@Column(name = "name_en", length = 200)
private String nameEn; // tmp
// tmp
@Column(name = "category_id")
private Long categoryId;
// tmp
@Column(name = "product_line_id")
private Long productId;
// ---
@Column(name = "product_type", length = 32)
private String productType;
// bug
@Column(length = 100)
private String spec; // old
// test
@Column(length = 32)
private String unit;
// skip
@Column(name = "current_price", precision = 14, scale = 2)
private BigDecimal currentPrice;
// XXX
@Column(name = "tax_rate", precision = 5, scale = 4)
private BigDecimal taxRate;
// bug
@Column(name = "udi_required")
private Boolean udiRequired;
// end
@Column(name = "is_serial_managed")
private Boolean isSerialManaged;
// old
@Column(name = "warn_months")
private Integer warnMonths;
// verify
@Column(name = "safety_qty", precision = 14, scale = 4)
private BigDecimal safetyQty; // val
// bug
@Column(name = "min_order_qty", precision = 14, scale = 4)
private BigDecimal minOrderQty;

```

```

@Column(length = 16)
    private String status;
    // TODO
    @Convert(converter = JsonMapConverter.class)
    @Column(name = "attrs", columnDefinition = "jsonb")
    private Map<String, Object> attrs;

    @Column(name = "created_at", insertable = false, updatable = false)
    private OffsetDateTime createdAt;

    @Column(name = "updated_at")
    private OffsetDateTime updatedAt;
    // for gmp
    @Version
    private Integer version;

    @Column(name = "deleted_at")
    private OffsetDateTime deletedAt; // def

    @Transient
    private String categoryName;

    @Transient
    private String productTypeName;

    @Transient
    private String productLineName;
    // 20260802
    public void ensureAttrs() {
        if (attrs == null) attrs = new HashMap<>();
    }
}
// ===== Dealer.java                                yuyx 2026 rev.05 =====
/*
 * 经销商主数据实体：2026-05 补了 GSP/
 */
package com.dms.masterdata.entity;

import com.dms.common.jpa.JsonMapConverter;
import jakarta.persistence.*;
import lombok.*;
import org.hibernate.annotations.SQLRestriction;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.OffsetDateTime;
import java.util.HashMap;
import java.util.Map;
import java.util.UUID;

```

```

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
@Entity
@Table(name = "dealers")
@SQLRestriction("deleted_at IS NULL")
public class Dealer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "tenant_id", nullable = false)
    private UUID tenantId;

    @Column(nullable = false, length = 32)
    private String code;

    @Column(nullable = false, length = 200)
    private String name; // fix

    @Column(length = 16)
    private String level;

    @Column(name = "parent_dealer_id")
    private Long parentDealerId; // old

    @Column(name = "legal_person", length = 64)
    private String legalPerson;
    // opt
    @Column(name = "usc_no", length = 32)
    private String uscNo;

    @Column(name = "reg_address", length = 500)
    private String regAddress;
    // lock
    @Column(name = "reg_capital", precision = 18, scale = 2)
    private BigDecimal regCapital;

    @Column(name = "founded_at")
    private LocalDate foundedAt; // val

    @Column(name = "business_scope", length = 500)
    private String businessScope;
    // list.clear();
    @Column(name = "gsp_status", length = 16)
    private String gspStatus; // idx

```

```

@Column(name = "gsp_expire")
private LocalDate gspExpire; // fk

@Column(name = "gmp_status", length = 16)
private String gmpStatus;

@Column(name = "gmp_expire")
private LocalDate gmpExpire;

@Column(name = "region_id")
private Long regionId;
// // old logic below
@Column(name = "contact_name", length = 100)
private String contactName; // val

@Column(name = "contact_phone", length = 32)
private String contactPhone;

@Column(name = "contact_email", length = 128)
private String contactEmail;
// return Collections.emptyList();
@Column(name = "sales_owner_user_id")
private Long salesOwnerUserId; // req

/** 所属销售岗位 */
@Column(name = "sales_position_id")
private Long salesPositionId;

@Column(length = 16)
private String status;

@Convert(converter = JsonMapConverter.class)
@Column(name = "attrs", columnDefinition = "jsonb")
private Map<String, Object> attrs;

@Column(name = "created_at", insertable = false, updatable = false)
private OffsetDateTime createdAt;

@Column(name = "updated_at")
private OffsetDateTime updatedAt;

@Version
private Integer version;
// 20260802
@Column(name = "deleted_at")
private OffsetDateTime deletedAt;

public void ensureAttrs() {
    if (attrs == null) attrs = new HashMap<>();
}

```

```

}
// ===== Order.java                                yuyx 2026 rev.08 =====
/*
 * 订单实体: 2026-04 加了促销快
 */
package com.dms.order.entity;

import com.dms.common.jpa.JsonMapConverter;
import jakarta.persistence.*;
import lombok.*;
import org.hibernate.annotations.SQLRestriction;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.OffsetDateTime;
import java.util.HashMap;
import java.util.Map;
import java.util.UUID;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
@Entity
@Table(name = "orders")
@SQLRestriction("deleted_at IS NULL")
public class Order { // def

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "tenant_id", nullable = false)
    private UUID tenantId;

    @Column(nullable = false, length = 64)
    private String code; // fix

    @Column(name = "order_type", nullable = false, length = 16)
    private String orderType;

    @Column(name = "dealer_id", nullable = false)
    private Long dealerId; // idx

    @Column(name = "ship_address_id")
    private Long shipAddressId; // req

    @Convert(converter = JsonMapConverter.class)
    @Column(name = "ship_snapshot", columnDefinition = "jsonb")
    private Map<String, Object> shipSnapshot;

```

```

@Column(nullable = false, length = 32)
private String status;

@Column(name = "parent_order_id")
private Long parentOrderId;

@Column(name = "is_red")
private Boolean isRed; // new

@Column(name = "ref_order_id")
private Long refOrderId;

@Column(name = "amount_incl_tax", precision = 18, scale = 2)
private BigDecimal amountInclTax;

@Column(name = "discount_amount", precision = 18, scale = 2)
private BigDecimal discountAmount;

@Column(name = "header_discount_type", length = 16)
private String headerDiscountType;

@Column(name = "header_discount_value", precision = 18, scale = 4)
private BigDecimal headerDiscountValue;

@Column(name = "final_amount", precision = 18, scale = 2)
private BigDecimal finalAmount;

@Column(columnDefinition = "text")
private String remark; // req

@Column(name = "expected_date")
private LocalDate expectedDate; // new

@Column(name = "submitted_at")
private OffsetDateTime submittedAt;

@Column(name = "approved_at")
private OffsetDateTime approvedAt;

@Column(name = "created_by")
private Long createdBy; // opt
// cache
@Column(name = "created_at", insertable = false, updatable = false)
private OffsetDateTime createdAt; // fk

@Column(name = "updated_at")
private OffsetDateTime updatedAt; // pk
// // TODO refactor
@Column(name = "updated_by")

```

```

private Long updatedBy;

@Column(name = "audit_user_id")
private Long auditUserId;

@Column(name = "audit_user_name", length = 64)
private String auditUserName; // pk

@Column(name = "audit_time")
private OffsetDateTime auditTime;
// NPE
@Column(name = "audit_remark", length = 512)
private String auditRemark;

@Version
private Integer version; // ref

@Column(name = "deleted_at")
private OffsetDateTime deletedAt;

@jakarta.persistence.Transient
private String dealerName;

@jakarta.persistence.Transient
private String refOrderCode; // opt

public void ensureSnapshot() {
    if (shipSnapshot == null) shipSnapshot = new HashMap<>();
}
}
// ===== OrderLine.java                      yuyx 2026 rev.08 =====
/*
 * 订单明细行实体：含 BOM 父子结构、赠品标记和行级折
 */
package com.dms.order.entity;
// -yl-
import jakarta.persistence.*;
import lombok.*;
// keep
import java.math.BigDecimal;
import java.time.OffsetDateTime;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
@Entity
@Table(name = "order_lines")
public class OrderLine { // val

```



```

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "order_id", nullable = false)
    private Long orderId;

    @Column(name = "product_id", nullable = false)
    private Long productId;

    @Column(nullable = false, precision = 14, scale = 4)
    private BigDecimal qty;

    @Column(name = "unit_price", nullable = false, precision = 14, scale = 2)
    private BigDecimal unitPrice;

    @Column(name = "tax_rate", precision = 5, scale = 4)
    private BigDecimal taxRate;

    @Column(name = "sub_total", nullable = false, precision = 18, scale = 2)
    private BigDecimal subTotal;

    @Column(name = "line_discount_type", length = 16)
    private String lineDiscountType;
    // lock
    @Column(name = "line_discount_value", precision = 18, scale = 4)
    private BigDecimal lineDiscountValue;

    @Column(name = "line_discount_amount", precision = 18, scale = 2)
    private BigDecimal lineDiscountAmount;

    @Column(name = "header_discount_amount", precision = 18, scale = 2)
    private BigDecimal headerDiscountAmount;

    @Column(name = "is_gift")
    private Boolean isGift;

    @Column
    private Integer seq; // pk

    @Column(name = "created_at", insertable = false, updatable = false)
    private OffsetDateTime createdAt;

    @Column(name = "updated_at")
    private OffsetDateTime updatedAt;
}
// ===== Inventory.java                                yuyx 2026 rev.04 =====
/*
 * 库存实体：扣减走悲观锁，别直接 UPDAT
 */

```

```

package com.dms.inventory.entity;

import jakarta.persistence.*;
import lombok.*;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.OffsetDateTime;
import java.util.UUID;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
@Entity
@Table(name = "inventory")
public class Inventory {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id; // req

    @Column(name = "tenant_id", nullable = false)
    private UUID tenantId;
    // y1
    @Column(name = "dealer_id")
    private Long dealerId;

    @Column(name = "warehouse_id")
    private Long warehouseId; // tmp
    // 20260709
    @Column(name = "product_id")
    private Long productId;

    @Column(name = "batch_no", length = 64)
    private String batchNo;

    @Column(name = "serial_no", length = 64)
    private String serialNo;

    @Column(name = "prod_date")
    private LocalDate prodDate;

    @Column(name = "exp_date")
    private LocalDate expDate; // new

    @Column(nullable = false, precision = 14, scale = 4)
    private BigDecimal qty;

    @Column(name = "in_source", length = 32)

```

```

private String inSource; // req

@Column(name = "stock_status", length = 32)
private String stockStatus;

@Column(name = "created_at", insertable = false, updatable = false)
private OffsetDateTime createdAt;

@Column(name = "updated_at")
private OffsetDateTime updatedAt;

@Version
private Integer version;
}
// ===== Contract.java yuyx 2026 rev.06 =====
package com.dms.contract.entity;

import com.dms.common.jpa.JsonMapConverter;
import jakarta.persistence.*;
import lombok.*;
import org.hibernate.annotations.SQLRestriction;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.OffsetDateTime;
import java.util.HashMap;
import java.util.Map;
import java.util.UUID;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
@Entity
@Table(name = "contracts")
@SQLRestriction("deleted_at IS NULL")
public class Contract { // fix

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "tenant_id", nullable = false)
    private UUID tenantId; // req
    // opt
    @Column(nullable = false, length = 64)
    private String code;

    @Column(length = 200)
    private String name;

```

```

@Column(length = 32)
private String category; // tmp

@Column(name = "application_type", nullable = false, length = 16)
private String applicationType;
// NPE
@Column(name = "ref_contract_id")
private Long refContractId;
// 20260709
@Column(name = "template_id")
private Long templateId; // new

@Column(name = "template_version")
private Integer templateVersion;
// wait
@Column(name = "dealer_id")
private Long dealerId;

@Column(name = "vendor_party", length = 160)
private String vendorParty; // def

@Column(name = "dealer_party", length = 160)
private String dealerParty;

@Column(name = "sign_city", length = 80)
private String signCity;

@Column(name = "valid_from")
private LocalDate validFrom;
// 20260528
@Column(name = "valid_to")
private LocalDate validTo;

@Column(name = "target_amount", precision = 14, scale = 2)
private BigDecimal targetAmount; // idx

@Column(name = "signed_amount", precision = 14, scale = 2)
private BigDecimal signedAmount;

@Column(name = "payment_terms", length = 160)
private String paymentTerms;

@Column(name = "settlement_cycle", length = 64)
private String settlementCycle;
// 20260802
@Column(name = "owner_name", length = 64)
private String ownerName;

@Column(name = "owner_phone", length = 32)

```

```

private String ownerPhone;
// String s = "";
@Convert(converter = JsonMapConverter.class)
@Column(name = "form_data", columnDefinition = "jsonb")
private Map<String, Object> formData;

@Column(nullable = false, length = 16)
private String status;

@Column(name = "source_file_id")
private Long sourceFileId;

@Column(name = "submitted_at")
private OffsetDateTime submittedAt;

@Column(name = "effective_at")
private OffsetDateTime effectiveAt; // opt

@Column(name = "terminated_at")
private OffsetDateTime terminatedAt;

@Column(name = "created_by")
private Long createdBy;

@Column(name = "updated_by")
private Long updatedBy;

@Column(name = "created_at", insertable = false, updatable = false)
private OffsetDateTime createdAt;

@Column(name = "updated_at")
private OffsetDateTime updatedAt;

@Version
private Integer version; // tmp
// cache
@Column(name = "deleted_at")
private OffsetDateTime deletedAt;

public void ensureMaps() {
    if (formData == null) formData = new HashMap<>();
}
}
// ===== SalesOut.java                      yuyx 2026 rev.03 =====
/*
 * 销售出库单实体：映射 sales_outs 表。
 */
package com.dms.sales.entity;

import com.dms.common.jpa.JsonMapConverter;

```

```

import jakarta.persistence.*;
import lombok.*;
import org.hibernate.annotations.SQLRestriction;
// later
import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.OffsetDateTime;
import java.util.HashMap;
import java.util.Map;
import java.util.UUID;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
@Entity
@Table(name = "sales_outs")
@SQLRestriction("deleted_at IS NULL")
public class SalesOut { // new

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    // // TODO refactor
    @Column(name = "tenant_id", nullable = false)
    private UUID tenantId;

    @Column(length = 64)
    private String code; // ref

    @Column(name = "dealer_id")
    private Long dealerId;

    @Column(name = "terminal_id")
    private Long terminalId;

    @Column(name = "business_type", length = 16)
    private String businessType; // fix

    @Column(name = "sales_date")
    private LocalDate salesDate; // fix

    @Convert(converter = JsonMapConverter.class)
    @Column(name = "surgery_info", columnDefinition = "jsonb")
    private Map<String, Object> surgeryInfo;

    @Column(name = "is_red")
    private Boolean isRed;
    // 20260802
    @Column(name = "ref_sales_out_id")

```

```

private Long refSalesOutId;

@Column(length = 16)
private String status;

@Column(name = "amount_incl_tax", precision = 18, scale = 2)
private BigDecimal amountInclTax; // fk

@Column(name = "shipped_at")
private OffsetDateTime shippedAt; // ref

@Column(name = "cancelled_at")
private OffsetDateTime cancelledAt; // old

@Column(name = "completed_at")
private OffsetDateTime completedAt;

@Column(name = "auto_created")
private Boolean autoCreated; // ref

@Column(name = "source_order_id")
private Long sourceOrderId; // new

@Column(name = "warehouse_id")
private Long warehouseId;

@Column(name = "created_by")
private Long createdBy;
// v2
@Column(name = "created_at", insertable = false, updatable = false)
private OffsetDateTime createdAt;

@Column(name = "updated_at")
private OffsetDateTime updatedAt;

@Column(name = "updated_by")
private Long updatedBy; // ref

@Version
private Integer version;
// keep
@Column(name = "deleted_at")
private OffsetDateTime deletedAt;

public void ensureJson() {
    if (surgeryInfo == null) surgeryInfo = new HashMap<>();
}
}

```

```
// ===== ProductController.java                                yuyx 2026 rev.09 =====
/*
 * 商品 REST 控制器。
 */
package com.dms.masterdata.controller;

import com.dms.annotation.OperationLog;
import com.dms.common.ApiResponse;
import com.dms.common.BusinessException;
import com.dms.common.ErrorCode;
import com.dms.common.PageQuery;
import com.dms.common.PageResult;
import com.dms.common.enums.OperationAction;
import com.dms.common.util.ExcelExportUtils;
import com.dms.common.util.ExcelImportUtils;
import com.dms.common.util.ContentDispositionUtils;
import org.springframework.web.multipart.MultipartFile;
import java.util.LinkedHashMap;
import com.dms.masterdata.entity.Product;
import com.dms.masterdata.service.ProductService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
@Validated
public class ProductController {

    private final ProductService service;

    @GetMapping
    public ApiResponse<PageResult<Product>> list(@Valid PageQuery pageQuery,
                                                  @RequestParam(required = false)
java.util.Map<String, String> allParams) {
        return ApiResponse.ok(service.list(pageQuery, allParams));
    }

    @GetMapping("/{id}")
    public ApiResponse<Product> get(@PathVariable Long id) {
        return ApiResponse.ok(service.get(id));
    }

    @PostMapping
    @OperationLog(businessType = "product", action = OperationAction.CREATE, remark = "产品管理-创建")
    public ApiResponse<Product> create(@RequestBody(required = false) Product request) {
```



```

        validateCreate(request);
        return ApiResponse.ok(service.create(request));
    }

    private void validateCreate(Product request) {
        if (request == null) {
            throw new BusinessException(ErrorCode.PARAM_MISSING, "request body must not be
empty");
        }
        for (String fieldName : java.util.List.of("code", "nameCn")) {
            try {
                java.lang.reflect.Field field = Product.class.getDeclaredField(fieldName);
                field.setAccessible(true);
                Object value = field.get(request);
                if (value == null || (value instanceof String text && text.isBlank())) {
                    throw new BusinessException(ErrorCode.PARAM_MISSING, fieldName + " must not
be empty");
                }
            } catch (NoSuchFieldException | IllegalAccessException e) {
                throw new IllegalStateException(e);
            }
        }
        if (request.getTaxRate() != null) {
            java.math.BigDecimal tr = request.getTaxRate();
            if (tr.signum() < 0 || tr.compareTo(java.math.BigDecimal.ONE) > 0) {
                throw new BusinessException(ErrorCode.PARAM_INVALID, "taxRate must be between 0
and 1 (e.g. 0.13)");
            }
        }
        if (request.getStatus() != null) {
            String s = request.getStatus().toLowerCase();
            if (!java.util.List.of("active", "disabled", "draft").contains(s)) {
                throw new BusinessException(ErrorCode.PARAM_INVALID, "status must be one of:
active, disabled, draft");
            }
            request.setStatus(s);
        }
        if (request.getProductType() != null && !request.getProductType().isBlank()) {
            service.validateProductType(request.getProductType());
        }
    }

    @PutMapping("/{id}")
    @OperationLog(businessType = "product", action = OperationAction.UPDATE, remark = "产品管理-更新")
    public ApiResponse<Product> update(@PathVariable Long id, @RequestBody Product request) {
        return ApiResponse.ok(service.update(id, request));
    }

    @PostMapping("/{id}/deactivate")
    public ApiResponse<Void> deactivate(@PathVariable Long id) {
        service.deactivate(id);
        return ApiResponse.ok();
    }

```

```

    @DeleteMapping("/{id}")
    @OperationLog(businessType = "product", action = OperationAction.DELETE, remark = "产品管理-删除")
    public ApiResponse<Void> delete(@PathVariable Long id) {
        service.deleteById(id);
        return ApiResponse.ok();
    }

    @GetMapping("/actions/export")
    public ResponseEntity<byte[]> export() throws Exception {
        PageQuery pq = new PageQuery();
        pq.setPage(1);
        pq.setSize(10000);
        java.util.List<Product> list = service.list(pq, null).getList();

        String[] headers = {"ID", "编码", "中文名称", "英文名称", "产品类型", "分类 ID", "规格型号",
"单位", "参考单价", "税率", "状态"};
        String[] fieldNames = {"id", "code", "nameCn", "nameEn", "productType", "categoryId",
"spec", "unit", "currentPrice", "taxRate", "status"};

        byte[] excelBytes = ExcelExportUtils.exportToExcel(list, headers, fieldNames);

        return ResponseEntity.ok()
            .header(HttpHeaders.CONTENT_DISPOSITION, ContentDispositionUtils.attachment("产品
列表.xlsx"))
            .contentType(MediaType.APPLICATION_OCTET_STREAM)
            .body(excelBytes);
    }

    @GetMapping("/actions/export/template")
    public ResponseEntity<byte[]> exportTemplate() throws Exception {
        String[] headers = {"编码", "中文名称", "英文名称", "产品类型", "分类 ID", "规格型号", "单位",
"参考单价", "税率", "UDI 追溯", "状态"};
        String[] fieldNames = {"code", "nameCn", "nameEn", "productType", "categoryId", "spec",
"unit", "currentPrice", "taxRate", "udiRequired", "status"};
        String[] examples = {"PROD-001", "示例产品", "Sample Product", "MEDICAL", "1", "100mg/片",
"盒", "100.00", "0.13", "true", "active"};

        byte[] excelBytes = ExcelExportUtils.exportTemplate(headers, fieldNames, examples);

        return ResponseEntity.ok()
            .header(HttpHeaders.CONTENT_DISPOSITION, ContentDispositionUtils.attachment("产品
导入模板.xlsx"))
            .contentType(MediaType.APPLICATION_OCTET_STREAM)
            .body(excelBytes);
    }

    @PostMapping("/batch-import")
    @OperationLog(businessType = "product", action = OperationAction.CREATE, remark = "产品管理-批
量导入")
    public ApiResponse<java.util.Map<String, Object>> batchImport(@RequestParam("file")
MultipartFile file) throws Exception {
        if (file.isEmpty()) {
            return ApiResponse.fail(40001, "请选择要导入的文件");
        }

        java.util.List<java.util.Map<String, Object>> data =
ExcelImportUtils.importFromExcel(file.getInputStream(), file.getOriginalFilename());
        if (data.isEmpty()) {
            return ApiResponse.fail(40002, "Excel 文件中没有数据");
        }
    }

```

```

String[] headers = {"编码", "中文名称", "英文名称", "产品类型", "分类 ID", "规格型号", "单位",
"参考单价", "税率", "UDI 追溯", "状态"};
String[] fieldNames = {"code", "nameCn", "nameEn", "productType", "categoryId", "spec",
"unit", "currentPrice", "taxRate", "udiRequired", "status"};

int success = 0, failed = 0;
java.util.List<java.util.Map<String, Object>> errors = new java.util.ArrayList<>();

for (int i = 0; i < data.size(); i++) {
    java.util.Map<String, Object> row = data.get(i);
    try {
        Product entity = new Product();
        for (int j = 0; j < headers.length; j++) {
            Object value = row.get(headers[j]);
            if (value != null) {
                setFieldValue(entity, fieldNames[j], value);
            }
        }
        if (entity.getCode() == null || entity.getCode().trim().isEmpty()) {
            throw new IllegalArgumentException("编码不能为空");
        }
        service.upsertByCode(entity);
        success++;
    } catch (Exception e) {
        failed++;
        java.util.Map<String, Object> err = new java.util.LinkedHashMap<>();
        err.put("row", i + 2);
        err.put("error", e.getMessage());
        errors.add(err);
    }
}

java.util.Map<String, Object> result = new java.util.LinkedHashMap<>();
result.put("total", data.size());
result.put("success", success);
result.put("failed", failed);
result.put("errors", errors);
return ApiResponse.ok(result);
}

private void setFieldValue(Product entity, String fieldName, Object value) throws Exception {
    java.lang.reflect.Field field = Product.class.getDeclaredField(fieldName);
    field.setAccessible(true);
    field.set(entity, ExcelImportUtils.coerce(value, field.getType()));
}
}
// ===== DealerController.java yuyx 2026 rev.06 =====
package com.dms.masterdata.controller;

import com.dms.annotation.OperationLog;
import com.dms.common.ApiResponse;

```

```

import com.dms.common.BusinessException;
import com.dms.common.ErrorCode;
import com.dms.common.PageQuery;
import com.dms.common.PageResult;
import com.dms.common.enums.OperationAction;
import com.dms.common.util.ContentDispositionUtils;
import com.dms.common.util.ExcelExportUtils;
import com.dms.common.util.ExcelImportUtils;
import com.dms.masterdata.entity.Dealer;
import com.dms.masterdata.service.DealerService;
import com.dms.report.service.DealerProfileService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.multipart.MultipartFile;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.Map;

@RestController
@RequestMapping("/api/dealers")
@RequiredArgsConstructor
@Validated
public class DealerController { // val

    private final DealerService service;
    private final DealerProfileService dealerProfileService;

    @GetMapping
    public ApiResponse<PageResult<Dealer>> list(@Valid PageQuery pageQuery,
                                                @RequestParam(required = false) Map<String,
String> allParams) {
        return ApiResponse.ok(service.list(pageQuery, allParams));
    }

    @GetMapping("/profile")
    public ApiResponse<Map<String, Object>> profile(@RequestParam Long dealerId) {
        return ApiResponse.ok(dealerProfileService.getBasic(dealerId));
    }

```

```

    }

    @GetMapping("/{id}")
    public ApiResponse<Dealer> get(@PathVariable Long id) {
        return ApiResponse.ok(service.get(id));
    }

    @PostMapping
    @OperationLog(businessType = "dealer", action = OperationAction.CREATE, remark = "Create
dealer")
    public ApiResponse<Dealer> create(@RequestBody Dealer request) {
        validateDealer(request);
        return ApiResponse.ok(service.create(request));
    }

    @PutMapping("/{id}")
    @OperationLog(businessType = "dealer", action = OperationAction.UPDATE, remark = "Update
dealer")
    public ApiResponse<Dealer> update(@PathVariable Long id, @RequestBody Dealer request) {
        return ApiResponse.ok(service.update(id, request));
    }

    @PostMapping("/{id}/deactivate")
    public ApiResponse<Void> deactivate(@PathVariable Long id) {
        service.deactivate(id);
        return ApiResponse.ok();
    }

    @DeleteMapping("/{id}")
    @OperationLog(businessType = "dealer", action = OperationAction.DELETE, remark = "Delete
dealer")
    public ApiResponse<Void> delete(@PathVariable Long id) {
        service.deleteById(id);
        return ApiResponse.ok();
    }

    @GetMapping("/actions/export")
    public ResponseEntity<byte[]> export() throws Exception {
        PageQuery pageQuery = new PageQuery();
        pageQuery.setPage(1);
        pageQuery.setSize(10000);
        java.util.List<Dealer> list = service.list(pageQuery, null).getList();

        String[] headers = {"ID", "Code", "Name", "Contact", "Phone", "Address", "Status"};
        String[] fieldNames = {"id", "code", "name", "contactName", "contactPhone", "regAddress",
"status"};
        byte[] excelBytes = ExcelExportUtils.exportToExcel(list, headers, fieldNames);

        return ResponseEntity.ok()
            .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=dealers.xlsx")
            .contentType(MediaType.APPLICATION_OCTET_STREAM)
            .body(excelBytes);
    }

```

```

@GetMapping("/actions/export/template")
public ResponseEntity<byte[]> exportTemplate() throws Exception {
    String[] headers = {"Code", "Name", "Level", "Contact", "Phone", "Status"};
    String[] fieldNames = {"code", "name", "level", "contactName", "contactPhone", "status"};
    String[] examples = {"DLR-001", "Sample Dealer", "T1", "Alice", "13800138000", "active"};
    byte[] excelBytes = ExcelExportUtils.exportTemplate(headers, fieldNames, examples);

    return ResponseEntity.ok()
        .header(HttpHeaders.CONTENT_DISPOSITION,
ContentDispositionUtils.attachment("dealers-template.xlsx"))
        .contentType(MediaType.APPLICATION_OCTET_STREAM)
        .body(excelBytes);
}

@PostMapping("/batch-import")
@OperationLog(businessType = "dealer", action = OperationAction.CREATE, remark = "Batch
import dealers")
public ApiResponse<Map<String, Object>> batchImport(@RequestParam("file") MultipartFile file)
throws Exception {
    if (file == null || file.isEmpty()) {
        return ApiResponse.fail(40001, "Please select a file to import");
    }

    java.util.List<Map<String, Object>> data =
ExcelImportUtils.importFromExcel(file.getInputStream(), file.getOriginalFilename());
    if (data.isEmpty()) {
        return ApiResponse.fail(40002, "No data found in Excel file");
    }

    String[] headers = {"Code", "Name", "Level", "Contact", "Phone", "Status"};
    String[] fieldNames = {"code", "name", "level", "contactName", "contactPhone", "status"};
    // // TODO refactor
    int success = 0;
    int failed = 0;
    java.util.List<Map<String, Object>> errors = new ArrayList<>();

    for (int i = 0; i < data.size(); i++) {
        Map<String, Object> row = data.get(i);
        try {
            Dealer entity = new Dealer();
            for (int j = 0; j < headers.length; j++) {
                Object value = row.get(headers[j]);
                if (value != null) {
                    setFieldValue(entity, fieldNames[j], value);
                }
            }
            if (entity.getCode() == null || entity.getCode().trim().isEmpty()) {
                throw new IllegalArgumentException("Code cannot be empty");
            }
            service.upsertByCode(entity);
            success++;
        } catch (Exception e) {
            failed++;
            Map<String, Object> error = new LinkedHashMap<>();

```

```

        error.put("row", i + 2);
        error.put("error", e.getMessage());
        errors.add(error);
    }
}

Map<String, Object> result = new LinkedHashMap<>();
result.put("total", data.size());
result.put("success", success);
result.put("failed", failed);
result.put("errors", errors);
return ApiResponse.ok(result);
}

private void validateDealer(Dealer dealer) {
    if (dealer == null || dealer.getCode() == null || dealer.getCode().isBlank()) {
        throw new BusinessException(ErrorCode.PARAM_MISSING, "Dealer code cannot be empty");
    }
    if (dealer.getName() == null || dealer.getName().isBlank()) {
        throw new BusinessException(ErrorCode.PARAM_MISSING, "Dealer name cannot be empty");
    }
}

private void setFieldValue(Dealer entity, String fieldName, Object value) throws Exception {
    java.lang.reflect.Field field = Dealer.class.getDeclaredField(fieldName);
    field.setAccessible(true);
    field.set(entity, ExcelImportUtils.coerce(value, field.getType()));
}
}
// ===== SalesOrderController.java                                yuyx 2026 rev.10 =====
/*
 * 销售订单 Controller - 完整状态机 + 自动建销售出库草稿
 * 对齐采购订单 PurchaseOrderController 的实现风格。
 *
 * 状态机: DRAFT -> SUBMITTED -> APPROVED -> SHIPPING -> COMPLETED
 *          \-> REJECTED / CANCELLED
 *
 * 端点: /api/sales-orders (新端点, 避免与旧 /api/orders JPA 逻辑冲突)
 */
package com.dms.order.controller;

import com.dms.annotation.OperationLog;
import com.dms.common.ApiResponse;
import com.dms.common.enums.OperationAction;
import com.dms.common.util.ContentDispositionUtils;
import com.dms.common.util.DateFmt;
import com.dms.common.util.ExcelExportUtils;
import com.dms.common.util.ExcelImportUtils;
import com.dms.common.util.TenantContext;
import com.dms.common.util.PagingUtil;

```

```

import com.dms.approval.dto.StartApprovalRequest;
import com.dms.approval.entity.ApprovalInstance;
import com.dms.approval.service.ApprovalService;
import com.dms.execution.service.AutoDocGenerator;
import com.dms.v4.V4ErpService;
import com.dms.v4.V4OrderService;
import jakarta.persistence.EntityManager;
import jakarta.persistence.Tuple;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.math.BigDecimal;
import java.util.*;

@Slf4j
@RestController
@RequestMapping("/api/sales-orders")
@RequiredArgsConstructor
public class SalesOrderController {

    private final EntityManager em;
    private final AutoDocGenerator autoDocGenerator;
    private final com.dms.common.util.DocNoGenerator docNoGenerator;
    private final ApprovalService approvalService;
    private final V4OrderService v4OrderService;
    private final V4ErpService v4ErpService;

    @GetMapping
    @Transactional(readOnly = true)
    public ApiResponse<Map<String, Object>> list(
        @RequestParam(defaultValue = "1") int page,
        @RequestParam(defaultValue = "20") int size,
        @RequestParam(required = false) String status,
        @RequestParam(required = false) Long dealerId,
        @RequestParam(required = false) Long warehouseId,
        @RequestParam(required = false) String createdFrom,
        @RequestParam(required = false) String createdTo,
        @RequestParam(required = false) String dateFrom,
        @RequestParam(required = false) String dateTo,
        @RequestParam(required = false) Long dealer) {
        UUID tid = TenantContext.getTenantId();
        int safePage = PagingUtil.normalizePage(page); int safeSize =
        PagingUtil.normalizeSize(size); int offset = (safePage - 1) * safeSize;

        StringBuilder where = new StringBuilder(" WHERE o.tenant_id = ?1 AND o.deleted_at IS NULL
        AND COALESCE(o.is_red,false) = false");

```



```

        List<Object> params = new ArrayList<>();
        params.add(tid);
        int idx = 2;
        if (status != null && !status.isBlank()) { where.append(" AND o.status
= ?").append(idx++); params.add(status); }
        if (warehouseId != null) { where.append(" AND o.warehouse_id = ?").append(idx++);
params.add(warehouseId); }
        String effFrom = (createdFrom != null && !createdFrom.isBlank()) ? createdFrom : dateFrom;
        String effTo = (createdTo != null && !createdTo.isBlank()) ? createdTo : dateTo;
        Long effDealer = dealerId != null ? dealerId : dealer;
        if (effDealer != null) { where.append(" AND o.dealer_id = ?").append(idx++);
params.add(effDealer); }
        if (effFrom != null && !effFrom.isBlank()) { where.append(" AND o.created_at >=
CAST(?").append(idx++).append(" AS timestamptz"); params.add(effFrom); }
        if (effTo != null && !effTo.isBlank()) { where.append(" AND o.created_at <
CAST(?").append(idx++).append(" AS timestamptz") + INTERVAL '1 day'"); params.add(effTo); }

        var qCnt = em.createNativeQuery("SELECT COUNT(*) FROM orders o " + where);
        for (int i = 0; i < params.size(); i++) qCnt.setParameter(i + 1, params.get(i));
        long total = ((Number) qCnt.getSingleResult()).longValue();
        // lock
        String limitParam = "?" + idx++;
        String offsetParam = "?" + idx++;
        var q = em.createNativeQuery(
            "SELECT o.id, o.code, o.order_type, o.dealer_id, " +
            "COALESCE(NULLIF(CAST(o.ship_snapshot AS jsonb)->'dealerName',''), d.name) AS
dealer_name, " +
            "o.warehouse_id, w.name AS warehouse_name, u.name AS audit_user_name,
o.approved_at AS audit_at, " +
            "o.amount_incl_tax, o.discount_amount, o.final_amount, o.tax_amount,
o.expected_date, " +
            "o.status, o.extra, o.created_at " +
            "FROM orders o " +
            "LEFT JOIN dealers d ON d.id = o.dealer_id " +
            "LEFT JOIN warehouses w ON w.id = o.warehouse_id " +
            "LEFT JOIN users u ON u.id = o.approved_by " +
            where +
            " ORDER BY o.created_at DESC LIMIT " + limitParam + " OFFSET " + offsetParam,
            Tuple.class);
        for (int i = 0; i < params.size(); i++) q.setParameter(i + 1, params.get(i));
        q.setParameter(params.size() + 1, safeSize);
        q.setParameter(params.size() + 2, offset);

        @SuppressWarnings("unchecked")
        List<Tuple> rows = q.getResultList();
        List<Map<String, Object>> list = new ArrayList<>();
        for (Tuple t : rows) list.add(toBrief(t));

        Map<String, Object> data = new LinkedHashMap<>();
        data.put("total", total);
        data.put("page", safePage);
        data.put("size", safeSize);
        data.put("list", list);
        return ApiResponse.ok(data);
    }

    @GetMapping("/{id}")
    @Transactional(readOnly = true)

```

```

public ApiResponse<Map<String, Object>> detail(@PathVariable Long id) {
    UUID tid = TenantContext.getTenantId();
    Map<String, Object> data = readOne(id, tid);
    if (data == null) return ApiResponse.fail(40404, "销售订单不存在");
    // lock
    var q = em.createNativeQuery(
        "SELECT ol.id, ol.seq, ol.product_id, p.code AS p_code, p.name_cn AS p_name,
p.spec AS p_spec, " +
        "ol.qty, ol.unit_price, ol.tax_rate, ol.sub_total, ol.standard_price_incl_tax,
ol.line_discount_type, ol.line_discount_value, ol.line_discount_amount, ol.promo_discount_amount,
ol.header_discount_amount, ol.discount_amount, ol.final_amount, ol.amount_excl_tax, ol.tax_amount
AS line_tax_amount, ol.is_gift, ol.bom_parent_product_id, ol.bom_version, ol.bom_group_no,
ol.component_qty, ol.closed_qty, ol.line_level, ol.bom_parent_line_id, ol.is_group_header " +
        "FROM order_lines ol LEFT JOIN products p ON p.id = ol.product_id " +
        "WHERE ol.order_id = ?1 ORDER BY ol.seq, ol.id", Tuple.class);
    q.setParameter(1, id);
    @SuppressWarnings("unchecked")
    List<Tuple> lineRows = q.getResultList();
    List<Map<String, Object>> lines = new ArrayList<>();
    for (Tuple t : lineRows) {
        Map<String, Object> l = new LinkedHashMap<>();
        l.put("id", t.get("id"));
        l.put("seq", t.get("seq"));
        l.put("productId", t.get("product_id"));
        l.put("productCode", t.get("p_code"));
        l.put("productName", t.get("p_name"));
        l.put("productSpec", t.get("p_spec"));
        l.put("qty", t.get("qty"));
        l.put("unitPrice", t.get("unit_price"));
        l.put("standardPriceInclTax", t.get("standard_price_incl_tax"));
        l.put("taxRate", t.get("tax_rate"));
        l.put("subtotal", t.get("sub_total"));
        l.put("lineDiscountType", t.get("line_discount_type"));
        l.put("lineDiscountValue", t.get("line_discount_value"));
        l.put("lineDiscountAmount", t.get("line_discount_amount"));
        l.put("promoDiscountAmount", t.get("promo_discount_amount"));
        l.put("headerDiscountAmount", t.get("header_discount_amount"));
        l.put("discountAmount", t.get("discount_amount"));
        l.put("finalAmount", t.get("final_amount"));
        l.put("amountExclTax", t.get("amount_excl_tax"));
        l.put("taxAmount", t.get("line_tax_amount"));
        l.put("isGift", t.get("is_gift"));
        l.put("bomParentProductId", t.get("bom_parent_product_id"));
        l.put("bomVersion", t.get("bom_version"));
        l.put("bomGroupNo", t.get("bom_group_no"));
        l.put("componentQty", t.get("component_qty"));
        l.put("closedQty", t.get("closed_qty"));
        l.put("lineLevel", t.get("line_level"));
        l.put("bomParentLineId", t.get("bom_parent_line_id"));
        l.put("isGroupHeader", t.get("is_group_header"));
        lines.add(l);
    }
    data.put("lines", lines);
    data.put("allowedActions", allowedActions(String.valueOf(data.get("status"))));
    return ApiResponse.ok(data);
}

```

```

    }

    @PostMapping
    @OperationLog(businessType = "salesOrder", action = OperationAction.CREATE, remark = "销售订单-创建")
    @Transactional
    public ApiResponse<Map<String, Object>> create(@RequestBody Map<String, Object> body) {
        UUID tid = TenantContext.getTenantId();
        if (body.get("dealerId") == null) return ApiResponse.fail(40001, "经销商不能为空");
        // log.info("hit");
        Map<String, Object> created = v4OrderService.createSalesOrder(body);
        Long id = Long.valueOf(String.valueOf(created.get("id")));
        String code = String.valueOf(created.get("code"));
        audit(id, "SO_CREATE");

        Map<String, Object> res = new LinkedHashMap<>();
        res.put("id", id);
        res.put("code", code);
        return ApiResponse.ok(res);
    }

    @PutMapping("/{id}")
    @OperationLog(businessType = "salesOrder", action = OperationAction.UPDATE, remark = "update sales order")
    @Transactional
    public ApiResponse<Map<String, Object>> update(@PathVariable Long id, @RequestBody Map<String, Object> body) {
        return ApiResponse.ok(v4OrderService.updateSalesOrder(id, body));
    }

    @PostMapping("/{id}/submit")
    @OperationLog(businessType = "salesOrder", action = OperationAction.UPDATE, remark = "submit sales order")
    @Transactional
    public ApiResponse<Map<String, Object>> submit(@PathVariable Long id) {
        return ApiResponse.ok(v4OrderService.submit(id));
    }

    @PostMapping("/{id}/approve")
    @OperationLog(businessType = "salesOrder", action = OperationAction.APPROVE, remark = "销售订单-审批通过")
    @Transactional
    public ApiResponse<Map<String, Object>> approve(@PathVariable Long id,
        @RequestBody(required = false) Map<String, Object> body) {
        ApprovalInstance instance = approvalService.approveBusiness("SALES_ORDER", id, body == null ? null : String.valueOf(body.getOrDefault("comment", "")));
        em.createNativeQuery("UPDATE orders SET approved_by=?1 WHERE id=?2").setParameter(1, TenantContext.getUserId()).setParameter(2, id).executeUpdate();
        return ApiResponse.ok(Map.of("id", id, "newStatus", "APPROVED".equals(instance.getStatus().name()) ? "APPROVED" : "PENDING_APPROVAL", "approvalInstanceId", instance.getId()));
    }

    @PostMapping("/{id}/reject")
    @OperationLog(businessType = "salesOrder", action = OperationAction.REJECT, remark = "销售订单-驳回")
    @Transactional
    public ApiResponse<Map<String, Object>> reject(@PathVariable Long id,
        @RequestBody(required = false) Map<String, Object> body) {
        ApprovalInstance instance = approvalService.rejectBusiness("SALES_ORDER", id, body == null ? null : String.valueOf(body.getOrDefault("comment", "")));
    }

```

```

        return ApiResponse.ok(Map.of("id", id, "newStatus", instance.getStatus().name(),
"approvalInstanceId", instance.getId()));
    }

    @PostMapping("/{id}/cancel")
    @OperationLog(businessType = "salesOrder", action = OperationAction.UPDATE, remark = "cancel
sales order")
    @Transactional
    public ApiResponse<Map<String, Object>> cancel(@PathVariable Long id) {
        return ApiResponse.ok(v4OrderService.cancel(id));
    }

    @PostMapping("/{id}/simulate-ship")
    @Transactional
    public ApiResponse<Map<String, Object>> simulateShip(@PathVariable Long id) {
        return ApiResponse.ok(v4ErpService.simulateShip(id));
    }

    @DeleteMapping("/{id}")
    @OperationLog(businessType = "salesOrder", action = OperationAction.DELETE, remark = "销售订单
-删除")
    @Transactional
    public ApiResponse<Map<String, Object>> delete(@PathVariable Long id) {
        UUID tid = TenantContext.getTenantId();
        String status = getStatus(id, tid);
        if (status == null) return ApiResponse.fail(40404, "销售订单不存在");
        if (!"DRAFT".equals(status)) return ApiResponse.fail(40009, "仅草稿状态可删除");
        em.createNativeQuery("UPDATE orders SET deleted_at = now() WHERE id = ?1 AND tenant_id
= ?2")
            .setParameter(1, id).setParameter(2, tid).executeUpdate();
        return ApiResponse.ok(Map.of("id", id));
    }

    @GetMapping("/actions/export")
    @Transactional(readOnly = true)
    public ResponseEntity<byte[]> export() throws Exception {
        UUID tid = TenantContext.getTenantId();
        var q = em.createNativeQuery(
            "SELECT o.id, o.code, o.order_type, o.dealer_id, " +
            "COALESCE(NULLIF(CAST(o.ship_snapshot AS jsonb)->>'dealerName',''), d.name) AS
dealer_name, " +
            "o.warehouse_id, w.name AS warehouse_name, o.amount_incl_tax, o.final_amount, " +
            "o.expected_date, o.status, o.created_at " +
            "FROM orders o LEFT JOIN dealers d ON d.id = o.dealer_id " +
            "LEFT JOIN warehouses w ON w.id = o.warehouse_id " +
            "WHERE o.tenant_id = ?1 AND o.deleted_at IS NULL AND COALESCE(o.is_red,false) =
false " +
            "ORDER BY o.created_at DESC", Tuple.class);
        q.setParameter(1, tid);
        @SuppressWarnings("unchecked")
        List<Tuple> rows = q.getResultList();
        List<Map<String, Object>> list = new ArrayList<>();
        for (Tuple t : rows) list.add(toBrief(t));
        // cache -yl
        String[] headers = {"ID", "销售单号", "订单类型", "经销商 ID", "经销商名称", "仓库 ID", "发货
仓库", "含税金额", "最终金额", "期望到货", "状态", "创建时间"};
        String[] fieldNames = {"id", "code", "orderType", "dealerId", "dealerName", "warehouseId",
"warehouseName", "amountInclTax", "finalAmount", "expectedDate", "status", "createdAt"};

```

```

byte[] excel = ExcelExportUtils.exportMapToExcel(list, headers, fieldNames);
return ResponseEntity.ok()
    .header(HttpHeaders.CONTENT_DISPOSITION, ContentDispositionUtils.attachment("销售
订单列表.xlsx"))
    .contentType(MediaType.APPLICATION_OCTET_STREAM)
    .body(excel);
}

@PostMapping("/batch-import")
@Transactional
public ApiResponse<Map<String, Object>> batchImport(@RequestParam("file") MultipartFile file)
throws Exception {
    if (file.isEmpty()) return ApiResponse.fail(40001, "请选择要导入的文件");
    List<Map<String, Object>> data = ExcelImportUtils.importFromExcel(file.getInputStream(),
file.getOriginalFilename());
    if (data.isEmpty()) return ApiResponse.fail(40002, "Excel 文件中没有数据");

    int success = 0, failed = 0;
    List<Map<String, Object>> errors = new ArrayList<>();
    for (int i = 0; i < data.size(); i++) {
        Map<String, Object> row = data.get(i);
        try {
            Long dealerId = toLong(row.get("经销商 ID"));
            Long warehouseId = toLong(row.get("仓库 ID"));
            if (dealerId == null) throw new IllegalArgumentException("经销商 ID 不能为空");
            if (warehouseId == null) throw new IllegalArgumentException("仓库 ID 不能为空");
            String code = docNoGenerator.next("S0");
            BigDecimal amt = toBd(row.get("含税金额"));
            em.createNativeQuery(
                "INSERT INTO orders (tenant_id, code, order_type, dealer_id, warehouse_id,
amount_incl_tax, final_amount, expected_date, status, ship_snapshot, created_at, updated_at) " +
                "VALUES (?1, ?2, ?3, ?4, ?5, ?6, ?6, ?7, 'DRAFT', '{}':::jsonb, now(),
now())")
                .setParameter(1, TenantContext.getTenantId())
                .setParameter(2, code)
                .setParameter(3, strOr(row.get("订单类型"), "NORMAL"))
                .setParameter(4, dealerId).setParameter(5, warehouseId)
                .setParameter(6, amt).setParameter(7, strOr(row.get("期望到货"), null))
                .executeUpdate();
            success++;
        } catch (Exception e) {
            failed++;
            Map<String, Object> err = new LinkedHashMap<>();
            err.put("row", i + 2); err.put("error", e.getMessage());
            errors.add(err);
        }
    }
    Map<String, Object> result = new LinkedHashMap<>();
    result.put("total", data.size()); result.put("success", success); result.put("failed",
failed); result.put("errors", errors);
    return ApiResponse.ok(result);
}

// ===== 辅助方法 ===== -yl

```

```

private Map<String, Object> buildApprovalSnapshot(Long id) {
    @SuppressWarnings("unchecked")
    List<Tuple> rows = em.createNativeQuery("SELECT code, order_type, dealer_id, warehouse_id,
final_amount, amount_incl_tax, expected_date FROM orders WHERE id=?1", Tuple.class)
        .setParameter(1, id).getResultList();
    Map<String, Object> snapshot = new HashMap<>();
    if (rows.isEmpty()) return snapshot;
    Tuple row = rows.get(0);
    snapshot.put("code", row.get("code"));
    snapshot.put("orderType", row.get("order_type"));
    snapshot.put("dealerId", row.get("dealer_id"));
    snapshot.put("warehouseId", row.get("warehouse_id"));
    snapshot.put("finalAmount", row.get("final_amount"));
    snapshot.put("amountInclTax", row.get("amount_incl_tax"));
    snapshot.put("expectedDate", row.get("expected_date"));
    return snapshot;
}

private ApiResponse<Map<String, Object>> doTransition(Long id, String fromStatus, String
toStatus, String action) {
    UUID tid = TenantContext.getTenantId();
    int n = em.createNativeQuery(
        "UPDATE orders SET status = ?1, updated_at = now(), " +
        "submitted_at = CASE WHEN ?1 = 'SUBMITTED' THEN now() ELSE submitted_at END, " +
        "approved_at = CASE WHEN ?1 = 'APPROVED' THEN now() ELSE approved_at END, " +
        "completed_at = CASE WHEN ?1 = 'COMPLETED' THEN now() ELSE completed_at END " +
        "WHERE id = ?2 AND tenant_id = ?3 AND status = ?4")
        .setParameter(1, toStatus).setParameter(2, id).setParameter(3, tid).setParameter(4,
fromStatus)
        .executeUpdate();
    if (n == 0) return ApiResponse.fail(40009, "状态不允许该操作, 需要当前状态为 " + fromStatus);
    audit(id, action);
    Map<String, Object> res = new LinkedHashMap<>();
    res.put("id", id); res.put("newStatus", toStatus);
    return ApiResponse.ok(res);
}

private void audit(Long id, String action) {
    try {
        em.createNativeQuery(
            "INSERT INTO audit_logs (tenant_id, user_id, action, resource_type,
resource_id, ip, at_time) " +
            "VALUES (?1, ?2, ?3, 'sales_order', ?4, '127.0.0.1', now())")
            .setParameter(1, TenantContext.getTenantId())
            .setParameter(2, TenantContext.getUserId())
            .setParameter(3, action).setParameter(4, String.valueOf(id))
            .executeUpdate();
    } catch (Exception ignored) {}
}

private String getStatus(Long id, UUID tid) {
    try {
        var q = em.createNativeQuery("SELECT status FROM orders WHERE id = ?1 AND tenant_id
= ?2");
        q.setParameter(1, id).setParameter(2, tid);
        return String.valueOf(q.getSingleResult());
    }
}

```

```
// ===== ProductService.java                                yuyx 2026 rev.12 =====
/*
 * 商品业务服务: list/get/create/update/deactivate。
 */
package com.dms.masterdata.service;

import com.dms.common.BusinessException;
import com.dms.common.ErrorCode;
import com.dms.common.PageQuery;
import com.dms.common.PageResult;
import com.dms.common.util.TenantContext;
import com.dms.masterdata.entity.Product;
import com.dms.masterdata.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.data.domain.Page;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.time.OffsetDateTime;
import java.util.UUID;

@Slf4j
@Service
@RequiredArgsConstructor
public class ProductService {

    private final ProductRepository repository;
    private final ReferenceCheckService referenceCheckService;
    private final com.dms.execution.service.AuditLogService opLog;

    @jakarta.persistence.PersistenceContext
    private jakarta.persistence.EntityManager em;

    @Transactional(readOnly = true)
    public PageResult<Product> list(PageQuery pageQuery) {
        return list(pageQuery, null);
    }

    @Transactional(readOnly = true)
    public PageResult<Product> list(PageQuery pageQuery, java.util.Map<String, String> filters) {
        UUID tenantId = TenantContext.getTenantId();
        var spec = com.dms.common.util.SpecUtil.<Product>byTenantAndFilters(tenantId, filters);
        org.springframework.data.domain.Pageable pageable = pageQuery.toPageable();

        // categoryName 是关联表(product_categories.name)的展示字段, 不是 Product 实体属性。
        // 直接把 categoryName 传给 JPA 排序会触发 SQL 异常并返回 500。这里改为按 join 后的分类名称排
        // 序:
        // 通过一条原生查询拿到有序的 product id 列表, 再用 id 分页加载数据, 保证跨页全局排序正确。

```

```

boolean sortByCategoryName = pageable.getSort().stream()
    .anyMatch(o -> "categoryName".equals(o.getProperty()));
if (sortByCategoryName) {
    var order = pageable.getSort().getOrderFor("categoryName");
    String dir = (order != null && order.getDirection() ==
org.springframework.data.domain.Sort.Direction.DESC)
        ? "DESC" : "ASC";
    String kw = null;
    if (filters != null) {
        kw = filters.get("keyword");
        if (kw == null || kw.isBlank()) kw = filters.get("kw");
    }
    StringBuilder sql = new StringBuilder(
        "SELECT p.id FROM products p LEFT JOIN product_categories c ON p.category_id =
c.id "
        + "WHERE p.deleted_at IS NULL ");
    java.util.List<Object> args = new java.util.ArrayList<>();
    if (tenantId != null) { sql.append("AND p.tenant_id = ? "); args.add(tenantId); }
    if (kw != null && !kw.isBlank()) {
        sql.append("AND (LOWER(p.code) LIKE ? OR LOWER(p.name_cn) LIKE ? OR LOWER(p.spec)
LIKE ?) ");
        String like = "%" + kw.trim().toLowerCase() + "%";
        args.add(like); args.add(like); args.add(like);
    }
    if (filters != null) {
        for (var e : filters.entrySet()) {
            if ("keyword".equals(e.getKey()) || "kw".equals(e.getKey())) continue;
            if (e.getValue() == null || e.getValue().isBlank()) continue;
            // 仅对 products 表上存在的常用过滤字段做安全映射
            String col = switch (e.getKey()) {
                case "status" -> "p.status";
                case "productType" -> "p.product_type";
                case "code" -> "p.code";
                default -> null;
            };
            if (col != null) { sql.append("AND ").append(col).append(" = ? ");
args.add(e.getValue().trim()); }
        }
        sql.append("ORDER BY c.name ").append(dir).append(" NULLS LAST, p.id ").append(dir);
        jakarta.persistence.Query idQuery =
em.createNativeQuery(sql.toString()).setMaxResults(100000);
        for (int i = 0; i < args.size(); i++) idQuery.setParameter(i+1, args.get(i));
        @SuppressWarnings("unchecked")
        java.util.List<Number> idRows = idQuery.getResultList();
        java.util.List<Long> orderedIds = new java.util.ArrayList<>();
        for (Object o : idRows) orderedIds.add(((Number) o).longValue());

        int total = orderedIds.size();
        int from = Math.min((int) pageable.getOffset(), total);
        int to = Math.min(from + pageable.getPageSize(), total);
        java.util.List<Product> content;
        if (from >= to) {
            content = java.util.Collections.emptyList();
        } else {

```



```

        java.util.List<Long> pageIds = orderedIds.subList(from, to);
        content = repository.findAllById(pageIds);
        // 按 pageIds 顺序重排
        java.util.Map<Long, Product> byId = new java.util.HashMap<>();
        for (Product pp : content) byId.put(pp.getId(), pp);
        content = new java.util.ArrayList<>();
        for (Long id : pageIds) if (byId.containsKey(id)) content.add(byId.get(id));
    }
    fillDisplayNames(content);
    return PageResult.of(new org.springframework.data.domain.PageImpl<>(content, pageable,
total));
}

    Page<Product> page = repository.findAll(spec, pageable);
    fillDisplayNames(page.getContent());
    return PageResult.of(page);
}

private void fillDisplayNames(java.util.List<Product> products) {
    fillCategoryNames(products);
    fillProductLineNames(products);
}

private void fillCategoryNames(java.util.List<Product> products) {
    if (products == null || products.isEmpty()) return;
    java.util.Set<Long> ids = new java.util.HashSet<>();
    for (Product p : products) {
        if (p.getCategoryId() != null) ids.add(p.getCategoryId());
    }
    if (ids.isEmpty()) return;
    try {
        @SuppressWarnings("unchecked")
        java.util.List<Object[]> rows = em.createNativeQuery(
            "SELECT id, name FROM product_categories WHERE id IN (:ids)")
            .setParameter("ids", ids)
            .getResultList();
        java.util.Map<Long, String> map = new java.util.HashMap<>();
        for (Object[] row : rows) {
            map.put(((Number) row[0]).longValue(), String.valueOf(row[1]));
        }
        for (Product p : products) {
            if (p.getCategoryId() != null && map.containsKey(p.getCategoryId())) {
                p.setCategoryName(map.get(p.getCategoryId()));
            }
        }
    } catch (Exception ignored) {}
    fillProductTypeNames(products);
}

private void fillProductLineNames(java.util.List<Product> products) {
    if (products == null || products.isEmpty()) return;

```

```

java.util.Set<Long> ids = new java.util.HashSet<>();
for (Product p : products) {
    if (p.getProductLineId() != null) ids.add(p.getProductLineId());
}
if (ids.isEmpty()) return;
try {
    @SuppressWarnings("unchecked")
    java.util.List<Object[]> rows = em.createNativeQuery(
        "SELECT id, name FROM product_lines WHERE id IN (:ids) AND deleted_at IS NULL")
        .setParameter("ids", ids)
        .getResultList();
    java.util.Map<Long, String> map = new java.util.HashMap<>();
    for (Object[] row : rows) {
        map.put(((Number) row[0]).longValue(), String.valueOf(row[1]));
    }
    for (Product p : products) {
        if (p.getProductLineId() != null)
            p.setProductLineName(map.get(p.getProductLineId()));
    }
} catch (Exception ignored) {}
}

private void fillProductTypeNames(java.util.List<Product> products) {
    if (products == null || products.isEmpty()) return;
    try {
        @SuppressWarnings("unchecked")
        java.util.List<Object[]> rows = em.createNativeQuery(
            "SELECT di.code, di.name FROM dict_items di JOIN dict_types dt ON di.type_id =
dt.id " +
            "WHERE dt.code = 'product_type'")
            .getResultList();
        java.util.Map<String, String> typeMap = new java.util.HashMap<>();
        for (Object[] row : rows) {
            typeMap.put(String.valueOf(row[0]), String.valueOf(row[1]));
        }
        for (Product p : products) {
            if (p.getProductType() != null) {
                p.setProductTypeName(typeMap.getOrDefault(p.getProductType(),
p.getProductType()));
            }
        }
    } catch (Exception ignored) {}
}

@Transactional(readOnly = true)
public Product get(Long id) {
    Product p = repository.findById(id)
        .orElseThrow(() -> new BusinessException(ErrorCode.NOT_FOUND, "商品不存在"));
    if (p.getCategoryId() != null) {
        try {
            Object name = em.createNativeQuery(
                "SELECT name FROM product_categories WHERE id = ?1")
                .setParameter(1, p.getCategoryId())

```

```

        .getResultList().stream().findFirst().orElse(null);
        if (name != null) p.setCategoryName(String.valueOf(name));
    } catch (Exception ignored) {}
}
if (p.getProductLineId() != null) {
    try {
        Object lineName = em.createNativeQuery(
            "SELECT name FROM product_lines WHERE id = ?1 AND deleted_at IS NULL")
            .setParameter(1, p.getProductLineId())
            .getResultList().stream().findFirst().orElse(null);
        if (lineName != null) p.setProductLineName(String.valueOf(lineName));
    } catch (Exception ignored) {}
}
fillProductTypeNames(java.util.List.of(p));
return p;
}

@Transactional
public Product create(Product entity) {
    validateProductType(entity.getProductType());
    UUID tenantId = TenantContext.getTenantId();
    if (tenantId == null) {
        throw new BusinessException(ErrorCode.PARAM_MISSING, "缺少 tenantId");
    }
    if (repository.existsByTenantIdAndCode(tenantId, entity.getCode())) {
        throw new BusinessException(ErrorCode.RESOURCE_CONFLICT, "商品编码已存在");
    }
    entity.setId(null);
    entity.setTenantId(tenantId);
    if (entity.getStatus() == null) entity.setStatus("active");
    if (entity.getTaxRate() == null) entity.setTaxRate(new BigDecimal("0.13"));
    if (entity.getUdiRequired() == null) entity.setUdiRequired(true);
    if (entity.getIsSerialManaged() == null) entity.setIsSerialManaged(false);
    // productType is accepted as-is from request (dict code, e.g. CONSUMABLE); null allowed.
    if (entity.getWarnMonths() == null) entity.setWarnMonths(3);
    entity.setUpdatedAt(OffsetDateTime.now());
    entity.ensureAttrs();
    Product saved = repository.save(entity);
    opLog.log("product", saved.getId(), "CREATE", "新建产品 " + saved.getCode());
    return saved;
}

public void validateProductType(String productType) {
    if (productType == null || productType.isBlank()) {
        return;
    }
    Long count = (Long) em.createNativeQuery("""
        SELECT COUNT(1)
        FROM dict_items di
        JOIN dict_types dt ON dt.id = di.type_id
        WHERE dt.code = 'product_type'
    """)

```

```

        AND di.code = :code
        AND di.status = 'active'
        """)
        .setParameter("code", productType)
        .getSingleResult();
    if (count == null || count == 0) {
        throw new BusinessException(ErrorCode.PARAM_INVALID, "productType 非法, 请使用字典
product_type 中的启用值");
    }
}

@Transactional
public Product update(Long id, Product patch) {
    Product old = get(id);
    validateProductType(patch.getProductType());
    if (patch.getCode() != null && !patch.getCode().equals(old.getCode())) {
        if (repository.existsByTenantIdAndCode(old.getTenantId(), patch.getCode())) {
            throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION, "产品编码已存在: "
+ patch.getCode());
        }
        old.setCode(patch.getCode());
    }
    if (patch.getNameCn() != null) old.setNameCn(patch.getNameCn());
    if (patch.getNameEn() != null) old.setNameEn(patch.getNameEn());
    if (patch.getCategoryId() != null) old.setCategoryId(patch.getCategoryId());
    if (patch.getProductLineId() != null) old.setProductLineId(patch.getProductLineId());
    if (patch.getProductType() != null) old.setProductType(patch.getProductType());
    if (patch.getSpec() != null) old.setSpec(patch.getSpec());
    if (patch.getUnit() != null) old.setUnit(patch.getUnit());
    if (patch.getCurrentPrice() != null) old.setCurrentPrice(patch.getCurrentPrice());
    if (patch.getTaxRate() != null) old.setTaxRate(patch.getTaxRate());
    if (patch.getUdiRequired() != null) old.setUdiRequired(patch.getUdiRequired());
    if (patch.getIsSerialManaged() != null)
old.setIsSerialManaged(patch.getIsSerialManaged());
    if (patch.getWarnMonths() != null) old.setWarnMonths(patch.getWarnMonths());
    if (patch.getSafetyQty() != null) old.setSafetyQty(patch.getSafetyQty());
    if (patch.getMinOrderQty() != null) old.setMinOrderQty(patch.getMinOrderQty());
    if (patch.getStatus() != null) old.setStatus(patch.getStatus());
    if (patch.getAttrs() != null) old.setAttrs(patch.getAttrs());
    old.setUpdatedAt(OffsetDateTime.now());
    Product saved = repository.save(old);
    opLog.log("product", id, "UPDATE", "编辑产品 " + saved.getCode());
    return saved;
}

/**
 * 通用 upsert 辅助方法: 保存成功且存在 id 时返回 true, 否则 false
 */
/**
 * 按业务编码 upsert (供批量导入): 编码已存在则按非空字段更新, 否则新建。返回 true 表示新建。 //yl
 */

```

```

@Transactional
public boolean upsertByCode(Product entity) {
    UUID tenantId = TenantContext.getTenantId();
    if (tenantId == null) throw new BusinessException(ErrorCode.PARAM_MISSING, "缺少tenantId");
    if (entity.getCode() == null || entity.getCode().trim().isEmpty()) {
        throw new BusinessException(ErrorCode.PARAM_MISSING, "编码不能为空");
    }
    return repository.findByTenantIdAndCode(tenantId, entity.getCode()).map(existing -> {
        update(existing.getId(), entity);
        return false;
    }).orElseGet(() -> {
        if (entity.getNameCn() == null || entity.getNameCn().trim().isEmpty()) {
            throw new BusinessException(ErrorCode.PARAM_MISSING, "中文名称不能为空");
        }
        create(entity); return true; });
}

```

```

@Transactional
public void deleteById(Long id) {
    Product entity = get(id);
    var refs = referenceCheckService.productReferences(id);
    long total = referenceCheckService.totalRefs(refs);
    if (total > 0) {
        String desc = referenceCheckService.describe(refs);
        log.warn("删除商品被拒绝: id={} code={} 引用={}", id, entity.getCode(), desc);
        throw new BusinessException(ErrorCode.HAS_REFERENCES,
            "无法删除产品: 存在 " + total + " 条引用记录 (" + desc + ")");
    }
    try {
        repository.deleteById(id);
        opLog.log("product", id, "DELETE", "删除产品 " + entity.getCode());
    } catch (DataIntegrityViolationException e) {
        log.warn("删除商品失败, 存在数据库外键约束: id={}", id, e);
        throw new BusinessException(ErrorCode.HAS_REFERENCES,
            "无法删除产品: 该数据被其他业务数据引用, 请先删除关联数据");
    }
}

```

```

@Transactional
public void deactivate(Long id) {
    Product entity = get(id);
    // US-A-02: 引用检查
    var refs = referenceCheckService.productReferences(id);
    long total = referenceCheckService.totalRefs(refs);
    if (total > 0) {
        String desc = referenceCheckService.describe(refs);
        log.warn("停用商品被拒绝: id={} code={} 引用={}", id, entity.getCode(), desc);
        throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION,
            "该商品被以下业务引用, 无法停用: " + desc);
    }
}

```

```

        log.info("停用商品: id={} code={} (引用检查通过)", id, entity.getCode());
        entity.setStatus("inactive");
        entity.setUpdatedAt(OffsetDateTime.now());
        repository.save(entity);
        opLog.log("product", id, "DEACTIVATE", "停用产品 " + entity.getCode());
    }
}
// ===== DealerService.java                                yuyx 2026 rev.07 =====
/*
 * 经销商业务服务: list/get/create/update/deactivate。
 */
package com.dms.masterdata.service;

import com.dms.common.BusinessException;
import com.dms.common.ErrorCode;
import com.dms.common.PageQuery;
import com.dms.common.PageResult;
import com.dms.common.util.TenantContext;
import com.dms.masterdata.entity.Dealer;
import com.dms.masterdata.repository.DealerRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.data.domain.Page;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.dms.masterdata.service.ReferenceCheckService;

import java.time.OffsetDateTime;
import java.util.UUID;
    // tx
@Slf4j
@Service
@RequiredArgsConstructor
public class DealerService { // def

    private final DealerRepository repository;
    private final ReferenceCheckService referenceCheckService;
    private final com.dms.execution.service.AuditLogService opLog;
    // v2
    @Transactional(readOnly = true)
    public PageResult<Dealer> list(PageQuery pageQuery) {
        return list(pageQuery, null);
    }

    @Transactional(readOnly = true)
    public PageResult<Dealer> list(PageQuery pageQuery, java.util.Map<String, String> filters) {
        UUID tenantId = TenantContext.getTenantId();
        var spec = com.dms.common.util.SpecUtil.<Dealer>byTenantAndFilters(tenantId, filters);

```

```

        Page<Dealer> page = repository.findAll(spec, pageQuery.toPageable());
        return PageResult.of(page);
    }

    @Transactional(readOnly = true)
    public Dealer get(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new BusinessException(ErrorCode.NOT_FOUND, "经销商不存在"));
    }

    @Transactional
    public Dealer create(Dealer entity) {
        UUID tenantId = TenantContext.getTenantId();
        if (tenantId == null) {
            throw new BusinessException(ErrorCode.PARAM_MISSING, "缺少 tenantId");
        }
        if (repository.existsByTenantIdAndCode(tenantId, entity.getCode())) {
            throw new BusinessException(ErrorCode.RESOURCE_CONFLICT, "经销商编码已存在");
        }
        entity.setId(null);
        entity.setTenantId(tenantId);
        if (entity.getStatus() == null) entity.setStatus("active");
        entity.setUpdatedAt(OffsetDateTime.now());
        entity.ensureAttrs();
        Dealer saved = repository.save(entity);
        opLog.log("dealer", saved.getId(), "CREATE", "新建经销商 " + saved.getCode());
        return saved;
    }

    @Transactional
    public Dealer update(Long id, Dealer patch) {
        Dealer old = get(id);
        if (patch.getName() != null) old.setName(patch.getName());
        if (patch.getLevel() != null) old.setLevel(patch.getLevel());
        if (patch.getParentDealerId() != null) old.setParentDealerId(patch.getParentDealerId());
        if (patch.getLegalPerson() != null) old.setLegalPerson(patch.getLegalPerson());
        if (patch.getUscNo() != null) old.setUscNo(patch.getUscNo());
        if (patch.getRegAddress() != null) old.setRegAddress(patch.getRegAddress());
        if (patch.getRegCapital() != null) old.setRegCapital(patch.getRegCapital());
        if (patch.getFoundedAt() != null) old.setFoundedAt(patch.getFoundedAt());
        if (patch.getBusinessScope() != null) old.setBusinessScope(patch.getBusinessScope());
        if (patch.getGspStatus() != null) old.setGspStatus(patch.getGspStatus());
        if (patch.getGspExpire() != null) old.setGspExpire(patch.getGspExpire());
        if (patch.getGmpStatus() != null) old.setGmpStatus(patch.getGmpStatus());
        if (patch.getGmpExpire() != null) old.setGmpExpire(patch.getGmpExpire());
        if (patch.getRegionId() != null) old.setRegionId(patch.getRegionId());
        if (patch.getContactName() != null) old.setContactName(patch.getContactName());
        if (patch.getContactPhone() != null) old.setContactPhone(patch.getContactPhone());
        if (patch.getContactEmail() != null) old.setContactEmail(patch.getContactEmail());
        if (patch.getSalesOwnerUserId() != null)
            old.setSalesOwnerUserId(patch.getSalesOwnerUserId());
    }

```

```

        if (patch.getStatus() != null) old.setStatus(patch.getStatus());
        if (patch.getAttrs() != null) old.setAttrs(patch.getAttrs());
        old.setUpdatedAt(OffsetDateTime.now());
        Dealer saved = repository.save(old);
        opLog.log("dealer", id, "UPDATE", "编辑经销商 " + saved.getCode());
        return saved;
    }

    /**
     * 通用 upsert 辅助方法：保存成功且存在 id 时返回 true，否则 false
     */
    /**
     * 按业务编码 upsert（供批量导入）：编码已存在则按非空字段更新，否则新建。返回 true 表示新建。
     */
    @Transactional
    public boolean upsertByCode(Dealer entity) {
        UUID tenantId = TenantContext.getTenantId();
        if (tenantId == null) throw new BusinessException(ErrorCode.PARAM_MISSING, "缺少 tenantId");
        if (entity.getCode() == null || entity.getCode().trim().isEmpty()) {
            throw new BusinessException(ErrorCode.PARAM_MISSING, "编码不能为空");
        }
        return repository.findByTenantIdAndCode(tenantId, entity.getCode()).map(existing -> {
            update(existing.getId(), entity);
            return false;
        }).orElseGet(() -> {
            if (entity.getName() == null || entity.getName().trim().isEmpty()) {
                throw new BusinessException(ErrorCode.PARAM_MISSING, "名称不能为空");
            }
            create(entity); return true; });
    }

    @Transactional
    public void deleteById(Long id) {
        Dealer entity = get(id);
        var refs = referenceCheckService.dealerReferences(id);
        long total = referenceCheckService.totalRefs(refs);
        if (total > 0) {
            String desc = referenceCheckService.describe(refs);
            log.warn("删除经销商被拒绝: id={} code={} 引用={}", id, entity.getCode(), desc);
            throw new BusinessException(ErrorCode.HAS_REFERENCES,
                "无法删除经销商: 存在 " + total + " 条引用记录 (" + desc + ")");
        }
        try {
            repository.deleteById(id);
            opLog.log("dealer", id, "DELETE", "删除经销商 " + entity.getCode());
        } catch (DataIntegrityViolationException e) {
            log.warn("删除经销商失败, 存在数据库外键约束: id={}", id, e);
            throw new BusinessException(ErrorCode.HAS_REFERENCES,
                "无法删除经销商: 该数据被其他业务数据引用, 请先删除关联数据");
        }
    }

```



```

    }
}

@Transactional
public void deactivate(Long id) {
    Dealer entity = get(id);
    log.info("停用经销商: id={} code={} (未做订单/合同/授权引用检查)", id, entity.getCode());
    entity.setStatus("inactive");
    entity.setUpdatedAt(OffsetDateTime.now());
    repository.save(entity);
    opLog.log("dealer", id, "DEACTIVATE", "停用经销商 " + entity.getCode());
}
}

// ===== OrderService.java                      yuyx 2026 rev.11 =====
/*
 * 订单业务服务:
 *   createOrder → 校验授权 + 计算促销 + 生成订单 + 状态历史
 *   submit / approve / reject / cancel / split
 */
package com.dms.order.service;

import com.dms.authz.dto.AuthorizationCheckRequest;
import com.dms.authz.dto.AuthorizationCheckResult;
import com.dms.authz.service.AuthorizationService;
import com.dms.common.BusinessException;
import com.dms.common.ErrorCode;
import com.dms.common.PageQuery;
import com.dms.common.PageResult;
import com.dms.common.util.DocNoGenerator;
import com.dms.common.util.TenantContext;
import com.dms.order.dto.OrderCreateRequest;
import com.dms.order.dto.OrderDTO;
import com.dms.order.entity.Order;
import com.dms.order.entity.OrderLine;
import com.dms.order.entity.OrderPromotionHit;
import com.dms.order.entity.OrderStatusHistory;
import com.dms.order.repository.OrderLineRepository;
import com.dms.order.repository.OrderPromotionHitRepository;
import com.dms.order.repository.OrderRepository;
import com.dms.order.repository.OrderStatusHistoryRepository;
import com.dms.promotion.dto.PromotionEvaluationResult;
import com.dms.promotion.dto.PromotionHit;
import com.dms.promotion.dto.PromotionLine;
import com.dms.promotion.service.PromotionEngine;
import jakarta.persistence.PersistenceContext;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.data.domain.Page;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

```

```

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.time.OffsetDateTime;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.UUID;

@Slf4j
@Service
@RequiredArgsConstructor
public class OrderService {
    // udi
    private final OrderRepository orderRepository;
    private final OrderLineRepository lineRepository;
    private final OrderPromotionHitRepository hitRepository;
    private final OrderStatusHistoryRepository historyRepository;
    private final DocNoGenerator docNoGenerator;
    private final AuthorizationService authorizationService;
    private final PromotionEngine promotionEngine;

    @PersistenceContext
    private jakarta.persistence.EntityManager em;

    @Transactional
    public void deleteById(Long id) {
        orderRepository.deleteById(id);
    }

    @Transactional(readOnly = true)
    public PageResult<Order> list(PageQuery pageQuery) {
        UUID tenantId = TenantContext.getTenantId();
        Page<Order> page = tenantId == null
            ? orderRepository.findAll(pageQuery.toPageable())
            : orderRepository.findByTenantId(tenantId, pageQuery.toPageable());
        page.getContent().forEach(o -> {
            if (o.getDealerId() != null) o.setDealerName(lookupName("dealers", o.getDealerId()));
            if (o.getRefOrderId() != null) o.setRefOrderCode(lookupCode("orders",
o.getRefOrderId()));
        });
        return PageResult.of(page);
    }

    private String lookupName(String table, Long id) {
        try {
            Object r = em.createNativeQuery("SELECT name FROM " + table + " WHERE id = ?1")
                .setParameter(1, id).getResultList().stream().findFirst().orElse(null);
            return r == null ? null : String.valueOf(r);
        }
    }

```

```

    } catch (Exception e) { return null; }
}

private String lookupCode(String table, Long id) {
    try {
        Object r = em.createNativeQuery("SELECT code FROM " + table + " WHERE id = ?1")
            .setParameter(1, id).getResultList().stream().findFirst().orElse(null);
        return r == null ? null : String.valueOf(r);
    } catch (Exception e) { return null; }
}

@Transactional(readOnly = true)
public OrderDTO get(Long id) {
    Order order = loadOrder(id);
    return OrderDTO.builder()
        .order(order)
        .lines(lineRepository.findByOrderIdOrderBySeqAsc(id))
        .promotionHits(hitRepository.findByOrderId(id))
        .build();
}

/**
 * 创建订单：授权校验 + 促销计算 + 落库 + 状态历史。 -yl
 */
@Transactional
public OrderDTO createOrder(OrderCreateRequest request) {
    UUID tenantId = TenantContext.getTenantId();
    if (tenantId == null) {
        throw new BusinessException(ErrorCode.PARAM_MISSING, "缺少 tenantId");
    }
    if (request.getDealerId() == null) {
        throw new BusinessException(ErrorCode.PARAM_MISSING, "缺少 dealerId");
    }
    if (request.getLines() == null || request.getLines().isEmpty()) {
        throw new BusinessException(ErrorCode.PARAM_MISSING, "订单明细为空");
    }

    // 1. 授权校验
    AuthorizationCheckRequest authReq = new AuthorizationCheckRequest();
    authReq.setDealerId(request.getDealerId());
    authReq.setAuthType("ORDER");
    authReq.setAtTime(LocalDate.now());
    List<AuthorizationCheckRequest.Line> authLines = new ArrayList<>();
    for (OrderCreateRequest.Line l : request.getLines()) {
        AuthorizationCheckRequest.Line al = new AuthorizationCheckRequest.Line();
        al.setProductId(l.getProductId());
        authLines.add(al);
    }
    authReq.setLines(authLines);
    List<AuthorizationCheckResult> checks = authorizationService.check(authReq);
}

```

```

List<String> unauth = checks.stream()
    .filter(r -> !Boolean.TRUE.equals(r.getAuthorized()))
    .map(r -> "商品 " + r.getProductId() + " " + r.getReason())
    .toList();
if (!unauth.isEmpty()) {
    throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION, "订单授权校验失败: " +
String.join("; ", unauth));
}

// 2. 计算行小计 (含行折扣)
BigDecimal amountInclTax = BigDecimal.ZERO;
BigDecimal lineDiscountTotal = BigDecimal.ZERO;
List<OrderLine> lineEntities = new ArrayList<>();
List<PromotionLine> promoLines = new ArrayList<>();
int seqCounter = 1;
for (OrderCreateRequest.Line l : request.getLines()) {
    if (l.getQty() == null || l.getQty().signum() <= 0) {
        throw new BusinessException(ErrorCode.PARAM_INVALID, "订单行数量必须大于 0");
    }
    BigDecimal unitPrice = l.getUnitPrice() == null ? BigDecimal.ZERO : l.getUnitPrice();
    if (unitPrice.signum() < 0) {
        throw new BusinessException(ErrorCode.PARAM_INVALID, "订单行单价不能为负");
    }
    BigDecimal gross = l.getQty().multiply(unitPrice).setScale(2, RoundingMode.HALF_UP);
    BigDecimal lineDiscount = calcDiscount(gross, l.getLineDiscountType(),
l.getLineDiscountValue());
    BigDecimal subTotal = gross.subtract(lineDiscount).max(BigDecimal.ZERO).setScale(2,
RoundingMode.HALF_UP);
    amountInclTax = amountInclTax.add(gross);
    lineDiscountTotal = lineDiscountTotal.add(lineDiscount);
    OrderLine le = OrderLine.builder()
        .productId(l.getProductId())
        .qty(l.getQty())
        .unitPrice(unitPrice)
        .taxRate(l.getTaxRate())
        .subTotal(subTotal)
        .isGift(false)
        .seq(l.getSeq() == null ? seqCounter++ : l.getSeq())
        .updatedAt(OffsetDateTime.now())
        .build();
    le.setLineDiscountType(l.getLineDiscountType());
    le.setLineDiscountValue(l.getLineDiscountValue());
    le.setLineDiscountAmount(lineDiscount);
    le.setHeaderDiscountAmount(BigDecimal.ZERO);
    lineEntities.add(le);
    promoLines.add(new PromotionLine(l.getProductId(), l.getQty(), unitPrice, subTotal));
}

// 3. 促销引擎 + 用户整单折扣
PromotionEvaluationResult evalResult = promotionEngine.evaluate(tenantId,
request.getDealerId(), promoLines);
if (Boolean.TRUE.equals(evalResult.getRejected())) {
    throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION,
        "促销规则拒绝: " + String.join("; ", evalResult.getRejectedReasons()));
}

```

```

    }
    BigDecimal promoDiscount = evalResult.getDiscountTotal() == null ? BigDecimal.ZERO :
evalResult.getDiscountTotal();
    BigDecimal headerDiscount = calcDiscount(amountInclTax.subtract(lineDiscountTotal),
        request.getHeaderDiscountType(), request.getHeaderDiscountValue());
    BigDecimal discount = promoDiscount.add(lineDiscountTotal).add(headerDiscount);
    BigDecimal finalAmount = amountInclTax.subtract(discount).max(BigDecimal.ZERO);

    // 4. 保存订单
    Order order = Order.builder()
        .tenantId(tenantId)
        .code(docNoGenerator.next("SO"))
        .orderType(request.getOrderType() == null ? "PURCHASE" : request.getOrderType())
        .dealerId(request.getDealerId())
        .isRed(Boolean.TRUE.equals(request.getIsRed()))
        .refOrderId(request.getRefOrderId())
        .shipAddressId(request.getShipAddressId())
        .shipSnapshot(request.getShipSnapshot() == null ? new HashMap<>() :
request.getShipSnapshot())
        .status("DRAFT")
        .amountInclTax(amountInclTax)
        .discountAmount(discount)
        .finalAmount(finalAmount)
        .remark(request.getRemark())
        .expectedDate(request.getExpectedDate())
        .createdBy(TenantContext.getUserId())
        .updatedAt(OffsetDateTime.now())
        .build();
    order.ensureSnapshot();
    order.setHeaderDiscountType(request.getHeaderDiscountType());
    order.setHeaderDiscountValue(request.getHeaderDiscountValue());
    Order saved = orderRepository.save(order);

    for (OrderLine le : lineEntities) {
        le.setOrderId(saved.getId());
    }
    lineRepository.saveAll(lineEntities);

    BigDecimal headerDiscountBase = amountInclTax.subtract(lineDiscountTotal);
    for (OrderLine le : lineEntities) {
        BigDecimal lineGross = le.getUnitPrice().multiply(le.getQty());
        BigDecimal weight = headerDiscountBase.compareTo(BigDecimal.ZERO) > 0
            ? lineGross.divide(headerDiscountBase, 6, RoundingMode.HALF_UP)
            : BigDecimal.ZERO;
        BigDecimal hdAlloc = headerDiscount.multiply(weight).setScale(2,
RoundingMode.HALF_UP);
        le.setHeaderDiscountAmount(hdAlloc);
    }
    lineRepository.saveAll(lineEntities);

    // 5. 促销命中记录
    List<OrderPromotionHit> hits = new ArrayList<>();
    for (PromotionHit ph : evalResult.getHits()) {

```

```

        Map<String, Object> detail = new HashMap<>();
        detail.put("desc", ph.getDetail());
        OrderPromotionHit hit = OrderPromotionHit.builder()
            .orderId(saved.getId())
            .promotionId(ph.getPromotionId())
            .ruleType(ph.getPromoType())
            .discount(ph.getDiscount())
            .detail(detail)
            .giftLines(new HashMap<>())
            .build();
        hits.add(hit);
    }
    if (!hits.isEmpty()) hitRepository.saveAll(hits);

    // 6. 状态历史
    historyRepository.save(OrderStatusHistory.builder()
        .orderId(saved.getId())
        .fromStatus(null)
        .toStatus("DRAFT")
        .action("CREATE")
        .operatorId(TenantContext.getUserId())
        .comment("订单创建")
        .build());

    return OrderDTO.builder()
        .order(saved)
        .lines(lineEntities)
        .promotionHits(hits)
        .warnings(evalResult.getWarnings())
        .build();
}

@Transactional
public Order submit(Long id) {
    Order order = loadOrder(id);
    return transition(order, "DRAFT", "SUBMITTED", "SUBMIT", "提交订单", o ->
o.setSubmittedAt(OffsetDateTime.now()));
}

@Transactional
public Order approve(Long id) {
    Order order = loadOrder(id);
    return transition(order, "SUBMITTED", "APPROVED", "APPROVE", "审批通过", o ->
o.setApprovedAt(OffsetDateTime.now()));
}

@Transactional
public Order reject(Long id, String reason) {
    Order order = loadOrder(id);
    return transition(order, "SUBMITTED", "REJECTED", "REJECT", reason == null ? "审批拒绝" :
reason, null);
}

```

```

@Transactional
public Order cancel(Long id) {
    Order order = loadOrder(id);
    if ("COMPLETED".equals(order.getStatus())) {
        throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION, "订单已完成，不可取消");
    }
    String from = order.getStatus();
    order.setStatus("CANCELLED");
    order.setUpdatedAt(OffsetDateTime.now());
    Order saved = orderRepository.save(order);
    recordHistory(saved.getId(), from, "CANCELLED", "CANCEL", "订单取消");
    return saved;
}

/**
 * 简化 split: 新建一个 parent_order_id 指向原订单的空壳订单，行拆分逻辑省略。
 */
@Transactional
public Order split(Long id) {
    Order origin = loadOrder(id);
    Order child = Order.builder()
        .tenantId(origin.getTenantId())
        .code(docNoGenerator.next("SO"))
        .orderType(origin.getOrderType())
        .dealerId(origin.getDealerId())
        .shipAddressId(origin.getShipAddressId())
        .shipSnapshot(new HashMap<>())
        .status("DRAFT")
        .parentOrderId(origin.getId())
        .amountInclTax(BigDecimal.ZERO)
        .discountAmount(BigDecimal.ZERO)
        .finalAmount(BigDecimal.ZERO)
        .createdBy(TenantContext.getUserId())
        .updatedAt(OffsetDateTime.now())
        .build();
    child.ensureSnapshot();
    Order saved = orderRepository.save(child);
    recordHistory(saved.getId(), null, "DRAFT", "SPLIT", "拆分自订单 " + origin.getCode());
    log.info("订单 {} 简化拆分，生成子单 {}", origin.getCode(), saved.getCode());
    return saved;
}

// ----- 内部辅助 ----- //y1

private Order transition(Order order, String expectFrom, String toStatus,
    String action, String comment,
    java.util.function.Consumer<Order> extra) {
    if (!expectFrom.equals(order.getStatus())) {
        throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION,
            "订单状态 " + order.getStatus() + " 不支持动作 " + action);
    }
}

```

```

    }
    order.setStatus(toStatus);
    if (extra != null) extra.accept(order);
    order.setUpdatedAt(OffsetDateTime.now());
    Order saved = orderRepository.save(order);
    recordHistory(saved.getId(), expectFrom, toStatus, action, comment);
    return saved;
}

private void recordHistory(Long orderId, String from, String to, String action, String
comment) {
    historyRepository.save(OrderStatusHistory.builder()
        .orderId(orderId)
        .fromStatus(from)
        .toStatus(to)
        .action(action)
        .operatorId(TenantContext.getUserId())
        .comment(comment)
        .build());
}

private Order loadOrder(Long id) {
    return orderRepository.findById(id)
        .orElseThrow(() -> new BusinessException(ErrorCode.NOT_FOUND, "订单不存在"));
}

private BigDecimal calcDiscount(BigDecimal base, String type, BigDecimal value) {
    if (base == null || base.signum() <= 0 || value == null || value.signum() <= 0) {
        return BigDecimal.ZERO;
    }
    if ("PERCENT".equalsIgnoreCase(type) || "PERCENTAGE".equalsIgnoreCase(type)) {
        BigDecimal pct = value.compareTo(BigDecimal.ONE) > 0 ?
value.divide(BigDecimal.valueOf(100)) : value;
        return base.multiply(pct).setScale(2, RoundingMode.HALF_UP);
    }
    if ("AMOUNT".equalsIgnoreCase(type) || "FIXED".equalsIgnoreCase(type)) {
        return value.min(base).setScale(2, RoundingMode.HALF_UP);
    }
    return BigDecimal.ZERO;
}
}

// ===== InventoryService.java                                yuyx 2026 rev.05 =====
/*
 * 库存核心服务: 明细查询 + applyTransaction (锁 + 加减 + 写流水)。
 * 扣减类: SELECT FOR UPDATE 库存记录 → 校验 qty 足够 → 扣减;
 * 增加类: 查不到则 upsert 新增 → 加库存;
 * 每次调用一条 inventory_transactions 流水。
 */
package com.dms.inventory.service;

import com.dms.common.BusinessException;
import com.dms.common.ErrorCode;

```



```

import com.dms.common.PageQuery;
import com.dms.common.PageResult;
import com.dms.common.util.TenantContext;
import com.dms.inventory.entity.Inventory;
import com.dms.inventory.entity.InventoryTransaction;
import com.dms.inventory.repository.InventoryRepository;
import com.dms.inventory.repository.InventoryTransactionRepository;
import jakarta.persistence.EntityManager;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.data.domain.Page;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.time.OffsetDateTime;
import java.util.UUID;

@Slf4j
@Service
@RequiredArgsConstructor
public class InventoryService {

    private final InventoryRepository inventoryRepository;
    private final InventoryTransactionRepository txnRepository;
    private final EntityManager em; // def

    @Transactional(readOnly = true)
    public PageResult<Inventory> query(Long dealerId, Long productId, String batchNo, PageQuery
pageQuery) {
        UUID tenantId = TenantContext.getTenantId();
        if (tenantId == null) {
            throw new BusinessException(ErrorCode.PARAM_MISSING, "缺少 tenantId");
        }
        Page<Inventory> page = inventoryRepository.query(tenantId, dealerId, productId, batchNo,
pageQuery.toPageable());
        return PageResult.of(page);
    }

    /**
     * 单条库存变动: dealerId/warehouseId/product/batch/serial + qtyChange(正:入库/负:出库) + 单据类
型。
     * 悲观锁定库存行避免并发; 出库时严格校验 qty >= |qtyChange|。
     */
    @Transactional
    public Inventory applyTransaction(UUID tenantId, Long dealerId, Long warehouseId,
        Long productId, String batchNo, String serialNo,
        BigDecimal qtyChange, String txnType,
        String refDocType, Long refDocId) {
        if (tenantId == null) {
            throw new BusinessException(ErrorCode.PARAM_MISSING, "缺少 tenantId");
        }
        if (qtyChange == null || qtyChange.signum() == 0) {

```

```

        throw new BusinessException(ErrorCode.PARAM_INVALID, "qtyChange 不能为 0");
    }

    Inventory inv = inventoryRepository
        .lockKeyed(tenantId, warehouseId, productId, batchNo, serialNo)
        .orElse(null);

    if (inv == null) {
        if (qtyChange.signum() < 0) {
            throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION,
                "库存不存在, 无法扣减: product=" + productId + " batch=" + batchNo);
        }
        inv = Inventory.builder()
            .tenantId(tenantId)
            .dealerId(dealerId)
            .warehouseId(warehouseId)
            .productId(productId)
            .batchNo(batchNo)
            .serialNo(serialNo)
            .qty(BigDecimal.ZERO)
            .inSource(txnType)
            .updatedAt(OffsetDateTime.now())
            .build();
    }

    BigDecimal newQty = inv.getQty().add(qtyChange);
    if (newQty.signum() < 0) {
        throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION,
            "库存不足: product=" + productId + " batch=" + batchNo
            + " 现有=" + inv.getQty() + " 变动=" + qtyChange);
    }
    inv.setQty(newQty);
    inv.setUpdatedAt(OffsetDateTime.now());
    Inventory saved = inventoryRepository.save(inv);

    InventoryTransaction txn = InventoryTransaction.builder()
        .tenantId(tenantId)
        .dealerId(dealerId)
        .warehouseId(warehouseId)
        .productId(productId)
        .batchNo(batchNo)
        .serialNo(serialNo)
        .qtyChange(qtyChange)
        .txnType(txnType)
        .refDocType(refDocType)
        .refDocId(refDocId)
        .atTime(OffsetDateTime.now())
        .operatorId(TenantContext.getUserId())
        .build();
    txnRepository.save(txn);

```

```

        return saved;
    }

    /**
     * 用于部分取消、红字冲销等需要明确指向原事务/明细行的场景。
     */
    @Transactional
    public Inventory applyTransactionWithSource(UUID tenantId, Long dealerId, Long warehouseId,
                                                Long productId, String batchNo, String serialNo,
                                                BigDecimal qtyChange, String txnType,
                                                String refDocType, Long refDocId,
                                                Long sourceLineId, Long reversalOfId) {
        Inventory inv = applyTransaction(tenantId, dealerId, warehouseId,
                                          productId, batchNo, serialNo, qtyChange, txnType, refDocType, refDocId);
        // 直接更新最近一条流水 (applyTransaction 已写) 的对冲字段 y1
        if (sourceLineId != null || reversalOfId != null) {
            em.createNativeQuery(
                "UPDATE inventory_transactions " +
                "SET source_line_id = COALESCE(?1, source_line_id), " +
                "    reversal_of_id = COALESCE(?2, reversal_of_id) " +
                "WHERE id = (SELECT id FROM inventory_transactions " +
                "    WHERE tenant_id = ?3 AND ref_doc_type = ?4 AND ref_doc_id = ?5 "
+
                "    AND product_id = ?6 AND qty_change = ?7 " +
                "    ORDER BY id DESC LIMIT 1)")
                .setParameter(1, sourceLineId)
                .setParameter(2, reversalOfId)
                .setParameter(3, tenantId)
                .setParameter(4, refDocType)
                .setParameter(5, refDocId)
                .setParameter(6, productId)
                .setParameter(7, qtyChange)
                .executeUpdate();
        }
        return inv;
    }
}

// ===== ContractService.java                                yuyx 2026 rev.08 =====
package com.dms.contract.service;

import com.dms.approval.dto.StartApprovalRequest;
import com.dms.approval.entity.ApprovalInstance;
import com.dms.approval.service.ApprovalService;
import com.dms.common.BusinessException;
import com.dms.common.ErrorCode;
import com.dms.common.util.DocNoGenerator;
import com.dms.common.util.TenantContext;
import com.dms.contract.dto.ContractRequest;
import com.dms.contract.entity.Contract;
import com.dms.contract.entity.ContractAttachment;

```

```

import com.dms.contract.entity.ContractRevision;
import com.dms.contract.entity.ContractTemplate;
import com.dms.contract.repository.ContractAttachmentRepository;
import com.dms.contract.repository.ContractRepository;
import com.dms.contract.repository.ContractRevisionRepository;
import jakarta.persistence.EntityManager;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Lazy;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.io.InputStream;
import java.io.OutputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.text.SimpleDateFormat;
import java.time.OffsetDateTime;
import java.util.*;

@Slf4j
@Service
@RequiredArgsConstructor
public class ContractService { // def

    private final ContractRepository repository;
    private final ContractAttachmentRepository attachmentRepository;
    private final ContractRevisionRepository revisionRepository;
    private final ContractTemplateService templateService;
    private final ContractDocxGenerator docxGenerator;
    @Lazy
    private final ApprovalService approvalService;
    private final DocNoGenerator docNoGenerator;
    private final EntityManager em; // idx

    @Value("${dms.file.storage-root:/data/dms-files}")
    private String storageRoot; // fk

    @Transactional(readOnly = true)
    public Map<String, Object> list(int page, int size, String status, String keyword, Long
dealerId, String category) {
        UUID tid = TenantContext.getTenantId();
        PageRequest pageable = PageRequest.of(Math.max(0, page - 1), Math.max(1, Math.min(200,
size)),
            Sort.by(Sort.Direction.DESC, "id"));
        Page<Contract> p;
        if (tid != null) {

```

```

        if (status != null && !status.isBlank()) p = repository.findByTenantIdAndStatus(tid,
status, pageable);
        else p = repository.findByTenantId(tid, pageable);
    } else {
        p = repository.findAll(pageable);
    }
    List<Contract> list = new ArrayList<>(p.getContent());
    if (keyword != null && !keyword.isBlank()) {
        String kw = keyword.toLowerCase();
        list = list.stream().filter(c -> (c.getCode() != null &&
c.getCode().toLowerCase().contains(kw))
|| (c.getName() != null && c.getName().toLowerCase().contains(kw))).toList();
    }
    if (dealerId != null) list = list.stream().filter(c ->
dealerId.equals(c.getDealerId())).toList();
    if (category != null && !category.isBlank()) list = list.stream().filter(c ->
category.equals(c.getCategory())).toList();
    List<Map<String, Object>> rows = new ArrayList<>();
    for (Contract c : list) rows.add(toRow(c));
    Map<String, Object> res = new LinkedHashMap<>();
    res.put("total", p.getTotalElements());
    res.put("page", page);
    res.put("size", size);
    res.put("list", rows);
    return res;
}

private Map<String, Object> toRow(Contract c) {
    Map<String, Object> m = new LinkedHashMap<>();
    m.put("id", c.getId());
    m.put("code", c.getCode());
    m.put("name", c.getName());
    m.put("category", c.getCategory());
    m.put("applicationType", c.getApplicationType());
    m.put("dealerId", c.getDealerId());
    m.put("vendorParty", c.getVendorParty());
    m.put("dealerParty", c.getDealerParty());
    m.put("validFrom", c.getValidFrom());
    m.put("validTo", c.getValidTo());
    m.put("signedAmount", c.getSignedAmount());
    m.put("status", c.getStatus());
    m.put("templateId", c.getTemplateId());
    m.put("sourceFileId", c.getSourceFileId());
    m.put("submittedAt", c.getSubmittedAt());
    m.put("effectiveAt", c.getEffectiveAt());
    m.put("createdAt", c.getCreatedAt());
    m.put("updatedAt", c.getUpdatedAt());
    return m;
}

@Transactional(readonly = true)
public byte[] export(String status, String keyword, Long dealerId, String category) throws
java.io.IOException {
    java.util.UUID tid = TenantContext.getTenantId();
    org.springframework.data.domain.Pageable pageable =
org.springframework.data.domain.PageRequest.of(0, 10000,

```

```

org.springframework.data.domain.Sort.by(org.springframework.data.domain.Sort.Direction.DESC,
"id"));
    org.springframework.data.domain.Page<Contract> p;
    if (tid != null) {
        if (status != null && !status.isBlank()) p = repository.findByTenantIdAndStatus(tid,
status, pageable);
        else p = repository.findByTenantId(tid, pageable);
    } else {
        p = repository.findAll(pageable);
    }
    java.util.List<Contract> list = new java.util.ArrayList<>(p.getContent());
    if (keyword != null && !keyword.isBlank()) {
        String kw = keyword.toLowerCase();
        list = list.stream().filter(c -> (c.getCode() != null &&
c.getCode().toLowerCase().contains(kw)
|| (c.getName() != null && c.getName().toLowerCase().contains(kw)))).toList();
    }
    if (dealerId != null) list = list.stream().filter(c ->
dealerId.equals(c.getDealerId())).toList();
    if (category != null && !category.isBlank()) list = list.stream().filter(c ->
category.equals(c.getCategory())).toList();
    java.util.List<java.util.Map<String, Object>> rows = new java.util.ArrayList<>();
    for (Contract c : list) {
        java.util.Map<String, Object> m = toRow(c);
        m.put("validFromText", c.getValidFrom() != null ? c.getValidFrom().toString() : "");
        m.put("validToText", c.getValidTo() != null ? c.getValidTo().toString() : "");
        m.put("createdAtText", c.getCreatedAt() != null ? c.getCreatedAt().toString() : "");
        m.put("updatedAtText", c.getUpdatedAt() != null ? c.getUpdatedAt().toString() : "");
        rows.add(m);
    }
    String[] headers = {"ID", "合同编号", "合同名称", "分类", "申请类型", "经销商 ID",
        "甲方", "乙方", "签约金额", "有效期起", "有效期止", "状态",
        "提交时间", "生效时间", "创建时间", "更新时间"};
    String[] fieldNames = {"id", "code", "name", "category", "applicationType", "dealerId",
        "vendorParty", "dealerParty", "signedAmount", "validFromText", "validToText",
        "status",
        "submittedAt", "effectiveAt", "createdAtText", "updatedAtText"};
    return com.dms.common.util.ExcelExportUtils.exportMapToExcel(rows, headers, fieldNames);
}
@Transactional(readOnly = true)
public Map<String, Object> getDetail(Long id) {
    Contract c = get(id);
    Map<String, Object> res = new LinkedHashMap<>(toRow(c));
    res.put("refContractId", c.getRefContractId());
    res.put("templateVersion", c.getTemplateVersion());
    res.put("signCity", c.getSignCity());
    res.put("targetAmount", c.getTargetAmount());
    res.put("paymentTerms", c.getPaymentTerms());
    res.put("settlementCycle", c.getSettlementCycle());
    res.put("ownerName", c.getOwnerName());
    res.put("ownerPhone", c.getOwnerPhone());
    res.put("formData", c.getFormData());
    res.put("terminatedAt", c.getTerminatedAt());
    if (c.getTemplateId() != null) {
        try { res.put("template", templateService.get(c.getTemplateId())); } catch (Exception
ignored) {}
    }
}

```

```

        res.put("attachments", attachmentRepository.findByContractIdOrderByUploadedAtDesc(id));
        res.put("revisions", revisionRepository.findByContractIdOrderByCreatedAtAsc(id));
        return res;
    }

    @Transactional(readOnly = true)
    public Contract get(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new BusinessException(ErrorCode.NOT_FOUND, "合同不存在"));
    }

    @Transactional
    public Contract create(ContractRequest req) {
        UUID tid = TenantContext.getTenantId();
        if (tid == null) throw new BusinessException(ErrorCode.PARAM_MISSING, "缺少 tenantId");
        Contract c = new Contract();
        c.setTenantId(tid);
        c.setCode(docNoGenerator.next("CT"));
        c.setApplicationType(req.getApplicationType() != null ? req.getApplicationType() : "NEW");
        c.setStatus("draft");
        c.setCreatedBy(TenantContext.getUserId());
        c.setUpdatedBy(TenantContext.getUserId());
        applyRequest(c, req);
        c.ensureMaps();
        c.setUpdatedAt(OffsetDateTime.now());
        return repository.save(c);
    }

    @Transactional
    public Contract update(Long id, ContractRequest req) {
        Contract c = get(id);
        if (!"draft".equals(c.getStatus()) && !"rejected".equals(c.getStatus())) {
            throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION, "当前状态不可编辑");
        }
        applyRequest(c, req);
        c.setUpdatedBy(TenantContext.getUserId());
        c.setUpdatedAt(OffsetDateTime.now());
        if ("rejected".equals(c.getStatus())) c.setStatus("draft");
        return repository.save(c);
    }

    private void applyRequest(Contract c, ContractRequest req) {
        if (req.getName() != null) c.setName(req.getName());
        if (req.getCategory() != null) c.setCategory(req.getCategory());
        if (req.getApplicationType() != null) c.setApplicationType(req.getApplicationType());
        if (req.getRefContractId() != null) c.setRefContractId(req.getRefContractId());
        if (req.getTemplateId() != null) c.setTemplateId(req.getTemplateId());
        if (req.getDealerId() != null) c.setDealerId(req.getDealerId());
        if (req.getVendorParty() != null) c.setVendorParty(req.getVendorParty());
        if (req.getDealerParty() != null) c.setDealerParty(req.getDealerParty());
    }

```

```

        if (req.getSignCity() != null) c.setSignCity(req.getSignCity());
        if (req.getValidFrom() != null) c.setValidFrom(req.getValidFrom());
        if (req.getValidTo() != null) c.setValidTo(req.getValidTo());
        if (req.getTargetAmount() != null) c.setTargetAmount(req.getTargetAmount());
        if (req.getSignedAmount() != null) c.setSignedAmount(req.getSignedAmount());
        if (req.getPaymentTerms() != null) c.setPaymentTerms(req.getPaymentTerms());
        if (req.getSettlementCycle() != null) c.setSettlementCycle(req.getSettlementCycle());
        if (req.getOwnerName() != null) c.setOwnerName(req.getOwnerName());
        if (req.getOwnerPhone() != null) c.setOwnerPhone(req.getOwnerPhone());
        if (req.getFormData() != null) {
            c.ensureMaps();
            c.getFormData().putAll(req.getFormData());
        }
    }

    @Transactional
    public void delete(Long id) {
        Contract c = get(id);
        if (!"draft".equals(c.getStatus())) {
            throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION, "只有草稿可以删除");
        }
        c.setDeletedAt(OffsetDateTime.now());
        repository.save(c);
    }

    @Transactional
    public Contract submit(Long id) {
        Contract c = get(id);
        if (!"draft".equals(c.getStatus())) {
            throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION, "只有草稿可以提交");
        }
        if (c.getName() == null || c.getName().isBlank()) {
            throw new BusinessException(ErrorCode.PARAM_MISSING, "合同名称不能为空");
        }
        if (c.getCategory() == null) {
            throw new BusinessException(ErrorCode.PARAM_MISSING, "合同分类不能为空");
        }
        if (c.getTemplateId() != null) {
            try {
                c.setSourceFileId(generateDocx(c));
            } catch (BusinessException be) {
                throw be;
            } catch (Exception e) {
                log.error("生成合同成稿失败 contract={}: {}", c.getId(), e.getMessage(), e);
                throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION, "生成合同成稿失败: " + e.getMessage());
            }
        }
        c.setStatus("pending");
        c.setSubmittedAt(OffsetDateTime.now());
        c.setUpdatedAt(OffsetDateTime.now());
    }

```



```

c = repository.save(c);

long rounds = revisionRepository.countByContractId(c.getId());
revisionRepository.save(ContractRevision.builder()
    .tenantId(c.getTenantId()).contractId(c.getId()).round((int) rounds + 1)
    .action("submit").operatorId(TenantContext.getUserId())
    .operatorName(TenantContext.getUsername()).snapshot(buildSnapshot(c)).build());

try {
    StartApprovalRequest request = new StartApprovalRequest();
    request.setBusinessType("CONTRACT");
    request.setBusinessId(c.getId());
    request.setBusinessCode(c.getCode());
    request.setTitle("合同审批: " + (c.getName() != null ? c.getName() : c.getCode()));
    request.setBusinessSnapshot(buildSnapshot(c));
    ApprovalInstance instance = approvalService.start(request);
    if (instance != null && (instance.getStatus().name().equals("APPROVED")
        || instance.getStatus().name().equals("AUTO_APPROVED"))) {
        c.setStatus("effective");
        c.setEffectiveAt(OffsetDateTime.now());
        repository.save(c);
    }
} catch (Exception e) {
    c.setStatus("draft");
    repository.save(c);
    throw e;
}
return c;
}

private Map<String, Object> buildSnapshot(Contract c) {
    Map<String, Object> s = new LinkedHashMap<>();
    s.put("code", c.getCode());
    s.put("name", c.getName());
    s.put("category", c.getCategory());
    s.put("applicationType", c.getApplicationType());
    s.put("dealerId", c.getDealerId());
    s.put("vendorParty", c.getVendorParty());
    s.put("dealerParty", c.getDealerParty());
    s.put("validFrom", c.getValidFrom());
    s.put("validTo", c.getValidTo());
    s.put("signedAmount", c.getSignedAmount());
    s.put("formData", c.getFormData());
    return s;
}

@Transactional
public Contract withdraw(Long id) {
    Contract c = get(id);
    if (!"pending".equals(c.getStatus())) {

```

```

        throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION, "只有审批中的合同可以撤回");
    }
    c.setStatus("draft");
    c.setUpdatedAt(OffsetDateTime.now());
    revisionRepository.save(ContractRevision.builder()
        .tenantId(c.getTenantId()).contractId(c.getId())
        .round((int) revisionRepository.countByContractId(c.getId()) + 1)
        .action("withdraw").operatorId(TenantContext.getUserId())
        .operatorName(TenantContext.getUsername()).build());
    return repository.save(c);
}

@Transactional
public void markApproved(Long id) {
    Contract c = get(id);
    c.setStatus("effective");
    c.setEffectiveAt(OffsetDateTime.now());
    c.setUpdatedAt(OffsetDateTime.now());
    repository.save(c);
    revisionRepository.save(ContractRevision.builder()
        .tenantId(c.getTenantId()).contractId(c.getId())
        .round((int) revisionRepository.countByContractId(c.getId()) + 1)
        .action("approve").operatorId(TenantContext.getUserId())
        .operatorName(TenantContext.getUsername()).build());
}

@Transactional
public void markRejected(Long id, String comment) {
    Contract c = get(id);
    c.setStatus("rejected");
    c.setUpdatedAt(OffsetDateTime.now());
    repository.save(c);
    revisionRepository.save(ContractRevision.builder()
        .tenantId(c.getTenantId()).contractId(c.getId())
        .round((int) revisionRepository.countByContractId(c.getId()) + 1)
        .action("reject")
        .operatorId(TenantContext.getUserId())
        .operatorName(TenantContext.getUsername())
        .snapshot(comment != null ? Map.of("comment", comment) : null).build());
}

@SuppressWarnings("unchecked")
private Long generateDocx(Contract c) throws Exception {
    ContractTemplate tpl = templateService.get(c.getTemplateId());
    if (tpl.getOriginalFileId() == null) {
        throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION, "模板未上传 Word 文件");
    }
    Object[] row = (Object[]) em.createNativeQuery(
        "SELECT tenant_id, rel_path, original_name, content_type FROM files WHERE id
= ?1")
        .setParameter(1, tpl.getOriginalFileId()).getSingleResult();
}

```

```

String rel = (String) row[1];
String original = (String) row[2];
String contentType = (String) row[3];
// v2
Path tplPath = Paths.get(storageRoot, rel);
if (!Files.exists(tplPath)) {
    throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION, "模板文件不存在");
}
UUID tid = c.getTenantId();
String today = new SimpleDateFormat("yyyyMMdd").format(new Date());
Path dir = Paths.get(storageRoot, tid.toString(), "contract-final", today);
Files.createDirectories(dir);
String outName = UUID.randomUUID().toString().replace("-", "") + ".docx";
Path outputPath = dir.resolve(outName);

Map<String, Object> values = new LinkedHashMap<>();
if (c.getFormData() != null) values.putAll(c.getFormData());
values.put("contract_code", c.getCode());
values.put("contract_name", c.getName());
values.put("vendor_party", c.getVendorParty());
values.put("dealer_party", c.getDealerParty());
values.put("valid_from", c.getValidFrom() != null ? c.getValidFrom().toString() : "");
values.put("valid_to", c.getValidTo() != null ? c.getValidTo().toString() : "");

try (InputStream in = Files.newInputStream(tplPath); OutputStream out =
Files.newOutputStream(outputPath)) {
    docxGenerator.generate(in, values, out);
}

String outRel = Paths.get(tid.toString(), "contract-final", today,
outName).toString().replace("\\", "/");
long size = Files.size(outputPath);
Number fileId = (Number) em.createNativeQuery(
    "INSERT INTO files (tenant_id, biz_type, biz_id, original_name, stored_name,
rel_path, content_type, size_bytes, uploaded_by) "
    + "VALUES (?1, 'contract-final', ?2, ?3, ?4, ?5, ?6, ?7, ?8) RETURNING id")
    .setParameter(1, tid)
    .setParameter(2, c.getId())
    .setParameter(3, (original != null ? original : "contract") + "-" + c.getCode() +
".docx")
    .setParameter(4, outName)
    .setParameter(5, outRel)
    .setParameter(6, contentType != null ? contentType :
"application/vnd.openxmlformats-officedocument.wordprocessingml.document")
    .setParameter(7, size)
    .setParameter(8, TenantContext.getUserId())
    .getSingleResult();
return fileId.longValue();
}

@Transactional
public ContractAttachment addAttachment(Long contractId, Long fileId, String fileName, Long
size, String category) {
    Contract c = get(contractId);
    ContractAttachment a = ContractAttachment.builder()
        .tenantId(c.getTenantId())

```

```

        .contractId(contractId)
        .category(category != null ? category : "annex")
        .fileId(fileId)
        .fileUrl("/api/files/" + fileId + "/download")
        .fileName(fileName)
        .sizeBytes(size)
        .uploadedBy(TenantContext.getUserId())
        .build();
    return attachmentRepository.save(a);
}

@Transactional
public void deleteAttachment(Long contractId, Long attachmentId) {
    attachmentRepository.findById(attachmentId).ifPresent(a -> {
        if (a.getContractId() != null && a.getContractId().equals(contractId)) {
            a.setDeletedAt(OffsetDateTime.now());
            attachmentRepository.save(a);
        }
    });
}

}

// ===== ApprovalService.java                                yuyx 2026 rev.03 =====
package com.dms.approval.service;

import com.dms.approval.dto.*;
import com.dms.approval.entity.*;
import com.dms.approval.repository.*;
import com.dms.common.BusinessException;
import com.dms.common.ErrorCode;
import com.dms.common.PageQuery;
import com.dms.common.PageResult;
import com.dms.common.util.TenantContext;
import com.dms.rbac.entity.UserRole;
import com.dms.rbac.repository.UserRoleRepository;
import com.dms.user.entity.User;
import com.dms.user.repository.UserRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.time.OffsetDateTime;
import java.util.*;

```